

REPUBLIC OF CAMEROON

Peace-work-fatherland



AFRICAN INSTITUTE OF COMPUTER
SCIENCES – CAMEROON PAUL BIYA
TECHNOLOGICAL CENTER OF
EXCELLENCE

P.O Box 13719 Yaounde

Tel: (237) 22 72 99 57 / 22 72 99 58

E-mail : contact@iaicameroun.com

Website : www.iaicameroun.com

REPUBLIQUE DU CAMEROUN

Paix-travail-patrie



REALIZE TECH

Tel: (237) 22 23 26 44 / 654486 985

Email: info@realize.com

Website: realizeinternship.onrender.com

INTERNSHIP REPORT

AI DRIVEN DECISION SUPPORT SYSTEM FOR INTELLIGENT DONOR MATCHING IN BLOOD DONATION NETWORKS

Internship carried out from 6th July to 30th September 2026 in view of obtaining a
Bachelor's degree (BSc) in Computer Sciences for the option: Software Engineering

Written by: ASAAH ANDU ONYA

Level III Software Engineering student at AICS Cameroon

Supervisors

Academic supervisor

Mme ONGUENE VANESSA

Lecturer at AICS Cameroon

Professional supervisor

Mr TANUE MONETTE

COO of REALIZE

2025 – 2026
Academic Year



DEDICATION

TO THE ASAAH'S FAMILY



ACKNOWLEDGEMENTS

Drafting this document would not have been possible without the contribution of some people who took upon themselves to see this work being accomplished. My gratitude goes to the following people:

- The Resident Representative of AICS Cameroon, **MR. ARMAND CLAUDE ABANDA**, for his support, words of encouragement and the different advices on how to approach situations.
- **Mr. TANUE MONETTE**, our professional supervisor for a cheerful welcome and a good working experience in the enterprise.
- **Mme. ONGUENE VANESSA**, our academic supervisor, for a great follow up and advices during the realization of this document.
- To all my teachers of **AICS CAMEROON** for the knowledge they impacted me with for me to be where I am today.
- To my parents, **Dr. ASAAH THOMAS** and **Mrs. ASAAH MEROLINE**, for their love, support, encouragement and advice.
- To all personnel at **REALIZE** for their encouragement and advice during the internship period
- To all my friends and classmates of the 2025-2026 batch
- To everyone far or near not cited above who contributed to the realization of this work or project.



CONTENT

DEDICATION	i
ACKNOWLEDGEMENTS	ii
CONTENT	iii
LIST OF FIGURES	iv
LIST OF TABLES	iv
ABBREVIATIONS	viii
ABSTRACT	ix
RESUME	x
GENERAL INTRODUCTION	1
PART ONE: INSERTION PHASE	2
PART TWO: THE TECHNICAL PHASE	15
FILE I. EXISTING SYSTEMS	17
FILE II: SPECIFICATION BOOK	25
FILE III: ANALYSIS FILE	38
FILE IV: CONCEPTION FILE	67
FILE V: REALIZATION FILE	88
FILE VI: TEST OF FUNCTIONALITIES	104
FILE VII: INSTALLATION AND USER GUIDE	109
GENERAL CONCLUSION	123
BIBLIOGRAPHY	xi
WEBOGRAPHY	xii
TABLE OF CONTENTS	xiii



LIST OF FIGURES

Figure 1.	<i>Geographical Location of REALIZE with coordinates: Latitude 3.828°N, Longitude 11.5021°E, Altitude 725m.....</i>	7
Figure 2.	<i>Functional Organization of REALIZE</i>	10
Figure 3.	<i>Gantt chart.....</i>	32
Figure 4.	<i>2TUP Diagram</i>	43
Figure 5.	<i>Use case diagram formalism.....</i>	46
Figure 6.	<i>General Use Case Diagram</i>	49
Figure 7.	<i>Consult Active Request Use Case Diagram.....</i>	50
Table 15.	<i>Authentication textual description</i>	52
Figure 8.	<i>Communication diagram formalism</i>	54
Figure 9.	<i>Authentication communication diagram</i>	56
Figure 10.	<i>Blood request communication diagram</i>	57
Figure 11.	<i>Sequence diagram formalism</i>	58
Figure 12.	<i>Authentication sequence diagram</i>	60
Figure 13.	<i>Blood request sequence diagram</i>	61
Figure 14.	<i>Activity diagram formalism</i>	62
Figure 15.	<i>Authentication activity diagram</i>	64
Figure 16.	<i>Submit Blood Request Activity Diagram</i>	65
Figure 17.	<i>Hardware diagram of system</i>	70
Figure 18.	<i>Physical Architecture of system</i>	71
Figure 19.	<i>MVC architecture pattern.....</i>	72
Figure 20.	<i>Class diagram formalism</i>	73
Figure 21.	<i>System Class Diagram</i>	76
Figure 22.	<i>Formalism of object diagram</i>	77
Figure 23.	<i>System Object Diagram.....</i>	79



Figure 24.	<i>State machine diagram formalism</i>	80
Figure 25.	<i>Account state machine diagram</i>	82
Figure 26.	<i>Blood Request State Machine Diagram</i>	83
Figure 27.	<i>Package diagram formalism</i>	84
Figure 28.	<i>System Package Diagram</i>	86
Figure 29.	<i>MVC (source: https://th.bing.com)</i>	92
Figure 30.	<i>MVVM (source: https://th.bing.com)</i>	93
Figure 31.	<i>N-tier architecture (source: https://th.bing.com)</i>	95
Figure 32.	<i>Deployment diagram formalism</i>	97
Figure 33.	<i>System Deployment Diagram</i>	99
Figure 34.	<i>Component diagram</i>	100
Figure 35.	<i>System Component Diagram</i>	102
Figure 36.	<i>Submit blood request functionality test</i>	107
Figure 37.	<i>Create campaign functionality test</i>	107
Figure 38.	<i>MongoDB Installation</i>	112
Figure 39.	<i>MongoDB MSI Installer</i>	112
Figure 40.	<i>Launching of Installation wizard</i>	113
Figure 41.	<i>Click on NEXT</i>	113
Figure 42.	<i>License agreement</i>	114
Figure 43.	<i>Select complete setup</i>	114
Figure 44.	<i>Run service network user</i>	115
Figure 45.	<i>Selecting MongoDB compass</i>	115
Figure 46.	<i>Start installation</i>	116
Figure 47.	<i>Complete installation</i>	116
Figure 48.	<i>Signup page</i>	117
Figure 49.	<i>Login page</i>	118
Figure 51.	<i>Hospital dashboard</i>	119



Figure 52. *New request page*120

Figure 53. *Searching for donors*121



LIST OF TABLES

Table 1.	<i>Hardware Resources of REALIZE</i>	11
Table 2.	<i>Software Resources of REALIZE</i>	12
Table 3.	<i>Existing Systems</i>	21
Table 4.	<i>Criticism of Existing Systems</i>	22
Table 5.	<i>Chronogram of activities</i>	31
Table 6.	<i>Hardware resources of the project</i>	33
Table 7.	<i>Software resources of the project</i>	34
Table 8.	<i>Human resources of the project</i>	35
Table 9.	<i>Total project estimated cost</i>	35
Table 10.	<i>List of Participants</i>	36
Table 11.	<i>Structural and Behavioural Diagrams</i>	41
Table 12.	<i>Difference between UML and MERISE</i>	42
Table 13.	<i>Formalism of use case Diagram</i>	47
Table 14.	<i>Identification of actors and their use cases</i>	48
Table 16.	<i>Submit Blood Request Textual description</i>	53
Table 17.	<i>Formalism of the communication diagram</i>	55
Table 18.	<i>Formalism of sequence diagram</i>	59
Table 19.	<i>Formalism of the activity diagram</i>	63
Table 20.	<i>Formalism of class diagram</i>	74
Table 21.	<i>Formalism of object Diagram</i>	78
Table 22.	<i>Formalism of state machine diagram</i>	81
Table 23.	<i>Formalism of Package Diagram</i>	85
Table 24.	<i>Used technologies</i>	96
Table 25.	<i>Formalism of deployment diagram</i>	98
Table 26.	<i>Formalism of a component diagram</i>	101



ABBREVIATIONS

Legend	Definition
2TUP	Two Track Unified Process;
AICS	African Institute of Computer Sciences;
DBMS	Database Management System;
IDE	Integrated Development Environment;
JSON	JavaScript Object Notation;
MERISE	<i>Méthode d'Etude et de Réalisation Informatique pour des Systèmes d'Entreprise;</i>
MVC	Model View Controller;
OMT	Object Modelling Technic;
OOSE	Object Oriented Software Engineering;
KYC	Know Your Customer



ABSTRACT

Blood donation in Cameroon faces challenges such as difficulty in identifying compatible blood donors, delays in accessing blood supplies, fragmented donor information, and limited coordination between donors, hospitals, and blood banks. These challenges can lead to critical shortages, delayed transfusions, and increased risks for patients requiring urgent blood. This project aims to design and implement a mobile-based AI-driven system to ease the process by which hospitals or blood banks search and find compatible donors especially for critical conditions. BloodBridge uses (OpenAI), an artificial intelligence to analyze key factors such as blood group compatibility, donor availability, location, donation history, and patient requirements to intelligently prioritize suitable donors. The system provides a digital platform that connects eligible blood donors with healthcare facilities, enabling better awareness and improving access to compatible blood. It also incorporates real-time notifications, and messaging to ease reach and facilitate communication. The platform leverages modern technologies, including ReactNative and Next.js for the user interface, Node.js for backend services, and MongoDB for data storage, with open AI supporting intelligent donor matching and decision-making. BloodBridge is expected to reduce the time required to identify and contact suitable donors. By combining artificial intelligence with digital healthcare infrastructure, BloodBridge represents a significant step toward building a more responsive, reliable, and accessible blood donation ecosystem in Cameroon, while providing a scalable model that can be adapted to other African healthcare systems.

Keywords:

Blood Donation, Artificial intelligence, Donor matching, Healthcare, Mobile application, Communication.



RESUME

La donation de sang au Cameroun fait face à des défis tels que la difficulté à identifier des donneurs de sang compatibles, les retards dans l'accès aux réserves de sang, les informations fragmentées sur les donneurs et la coordination limitée entre les donneurs, les hôpitaux et les banques de sang. Ces défis peuvent entraîner des pénuries critiques, des retards de transfusion et des risques accrus pour les patients nécessitant du sang en urgence. Ce projet vise à concevoir et à mettre en œuvre un système mobile basé sur l'intelligence artificielle pour faciliter le processus par lequel les hôpitaux ou les banques de sang recherchent et trouvent des donneurs compatibles, en particulier pour les conditions critiques. BloodBridge utilise (OpenAI), une intelligence artificielle, pour analyser des facteurs clés tels que la compatibilité des groupes sanguins, la disponibilité des donneurs, la localisation, les antécédents de don et les exigences des patients afin de prioriser intelligemment les donneurs appropriés. Le système fournit une plateforme numérique qui met en relation les donneurs de sang éligibles avec les établissements de santé, permettant une meilleure sensibilisation et améliorant l'accès à un sang compatible. Il intègre également des notifications en temps réel et une messagerie pour faciliter la portée et la communication. La plateforme s'appuie sur des technologies modernes, notamment ReactNative et Next.js pour l'interface utilisateur, Node.js pour les services backend et MongoDB pour le stockage des données, avec OpenAI soutenant la mise en correspondance intelligente des donneurs et la prise de décision. BloodBridge devrait réduire le temps nécessaire pour identifier et contacter des donneurs appropriés. En combinant l'intelligence artificielle avec l'infrastructure numérique de santé, BloodBridge représente une étape significative vers la construction d'un écosystème de don de sang plus réactif, fiable et accessible au Cameroun, tout en fournissant un modèle évolutif pouvant être adapté à d'autres systèmes de santé africains.

Mots-clés :

Don de sang, Intelligence artificielle, Mise en correspondance des donneurs, Santé, Application mobile, Communication.



GENERAL INTRODUCTION

Blood donation plays a vital role in saving lives during emergencies, surgeries, childbirth complications, accidents, and medical treatments. However, blood donation management in Cameroon faces challenges such as difficulty in finding compatible donors, limited donor information, delays in communication, and poor coordination between donors and healthcare facilities. These challenges can lead to blood shortages and delays in providing patients with the blood they need.

BloodBridge is an IT project developed by a student of the African Institute of Computer Sciences (AICS) in Cameroon. It is an **AI-Driven Decision Support System for Intelligent Donor Matching in Blood Donation Networks**, designed to improve the process of identifying and connecting suitable blood donors with patients and healthcare facilities.

The system uses **OpenAI** to support intelligent donor matching and decision-making based on factors such as blood group compatibility, donor availability, location, and donation history. BloodBridge also provides digital tools for donor management, blood inventory monitoring and notifications.

The mobile interface is developed using **React Native**, while **Node.js** is used for backend services and **MongoDB** for data storage. By combining artificial intelligence with mobile technology, BloodBridge aims to reduce the time required to find suitable donors, improve blood availability, and contribute to a more efficient and reliable blood donation system in Cameroon.

This work is structured in six main phases. The first phase introduces the host company and describes the intern's integration process. The second defines the project specifications, identifying user needs and system constraints. The third phase concerns system analysis, presenting the methods and diagrams used for modelling. The fourth covers the design stage, detailing the conceptual and technical architecture of the proposed system. The fifth phase focuses on implementation, including the technologies and development techniques used. Finally, the sixth phase provides a user guide that presents, in a clear and graphical manner, the main functionalities of the application.



PART ONE: INSERTION PHASE



Preamble

This phase presents the details of how we were integrated in the host company, the company presentation and organization.

Overview

INTRODUCTION

- I. WELCOME AND INTEGRATION
- II. GENERAL PRESENTATION OF THE COMPANY
- III. ORGANISATION OF THE COMPANY
- IV. HARDWARE AND SOFTWARE RESOURCES OF THE COMPANY
- V. BRIEF REPRESENTATION OF THE PROJECT THEME

CONCLUSION



INTRODUCTION

The insertion phase was a period of about two (02) weeks, reserved for interns to discover and familiarize with the working environment. Here, interns got to know about the personnel of the establishment, the different hardware and software resources used, the different departments which constitute the enterprise, how the company functions both internally and externally. Thereafter, trainees were introduced to their work spaces and attributed a theme and an internship master, also called the professional supervisor. During this phase, the interns also had time to exchange ideas, most especially regarding the topics of preference for their internship, what they disliked, their beliefs and their experiences.



I. WELCOME AND INTEGRATION

The trainees arrived at the building hosting “REALIZE” on Wednesday, 1st July 2026 at 8:00 am and were welcomed by **Mr. TANUE Monette BUKWATI**, the Enterprise Frontend Developer, who introduced former to the work space and his collaborators. Later on, trainees were requested to make some findings on the possible internship themes that we can work on. Also, they discussed on the enterprise's working languages of preference and frameworks, and also shared his past working/professional experiences in different enterprises.

A working day at REALIZE is a none-hour shift, running from 07:30 am to 04:30 pm, from Mondays to Fridays, except on Public Holidays. Hence, the tasks of trainees in the company generally followed a predefine routine whereby they had to write at least five pages of the internship report daily, to complete the assigned task for the day, and to present any difficulties and challenges faced in the course of the day.

Work in the first week was devoted to verification of concept with basic HTML and CSS, to ensure that all the trainees and company staff were at the same level of understanding before work started. There were talks on project ideas, during which trainees were encouraged to propose project ideas on which they were instructed to carry out research on these ideas. The main reason for this approach was to avoid imposing working themes on the trainees.

During the second week, work proceeded with the verification of JavaScript concepts and with some exercises meant to test their reasoning and understanding of the concepts. Discussions were also held with our professional supervisor about the specification book and its different components. During such exchanges the interns were presented with the expectations of each component as well as to fine-tune the different project ideas.



II. GENERAL PRESENTATION OF THE COMPANY

1. History

Realize is a Cameroon based non-governmental tech start-up institute founded by Mr. NDELOGAKEH Daniel in 2022 which proposes IT solutions and empowers the use on new technologies in Cameroon.

2. Missions

The mission of Realize mainly relates to the empowerment and the perpetuation of Technology use. These missions include:

- Design and realize mobile-oriented software for companies and individuals
- Assist newborn companies for quick growth using computer sciences
- Provide training and certifications to improve qualified human resources in many fields of study
- Take an active part in the sustainable development of the world through innovative solutions and virtual reality.

3. Vision

At Realize, it is believed that the true potential of every organization and individual can be unlocked through the harmonious fusion of analysis, development, realization, and data analytics. The vision of every individual propels him or her forward, with guidance to make a lasting impact on the digital landscape and shape a future where innovation knows no boundaries.

4. Activities

The activities of Realize range from computer sciences, engineering, and training. We can outline the following:

- Conception realization, and hosting of websites
- Software development and maintenance
- Training in Software related fields
- Conception and realization of multimedia
- IT consulting and innovation

5. Geographical location



Figure 1. *Geographical Location of REALIZE with coordinates: Latitude 3.828°N, Longitude 11.5021°E, Altitude 725m*



III. ORGANIZATION OF THE ENTERPRISE

1. Administrative Organization of REALIZE

REALIZE is administratively organized as follows;

a. The General Management

This is the highest level of the company, which takes care of the following:

- Proper functioning of each department
- Define project strategies
- Provide leadership and guidance to the company's employees
- Makes critical decision that could affect the company's operations or reputation
- Severe as a point of contact for key stakeholders such as investors amongst others.

b. Human Resource Department

This department is in charge of the following:

- Recruitment and hiring
- Acts as a liaison between employees and management
- Manages employee compensation and benefits programs
- Develops and implements company policies and procedures, ensuring compliance with employment laws and regulations
- Manages administrative tasks related to employee records, data management, and HR systems.

c. Communication Department

This department is in charge of the following:

- Handles public relations activities, which involve managing the company's reputation and image
- Responsible for crafting and delivering messages to external stakeholders such as customers amongst others
- It fosters effective communication within the company
- They create and manage content across different platforms and channels.



d. Department of Financial Affaires

This department is in charge of the following:

- Responsible for developing and managing the company's financial plans and budgets
- Prepare and present accurate and timely financial reports to management, stakeholders, and regulatory authorities
- Responsible for managing costs and expenses within the organization
- Ensures compliance with tax laws and regulations.

e. Technical Department

This department is in charge of the following:

- Managing the company's technological infrastructure, including networks and hardware
- Play a role in managing and maintaining the company's data
- Implementing and maintaining cybersecurity measures to protect the company's digital assets from potential threats
- Interacts with technology vendors and manages relationships with external service providers.

f. Software Engineering Department

This department is in charge of the following:

- Primarily responsible for developing software applications and systems
- Responsible for maintaining and supporting software applications throughout their lifecycle
- Evaluation and realization of projects.



2. Functional Organization of REALIZE

The functional branch of REALIZE is organized as follows;

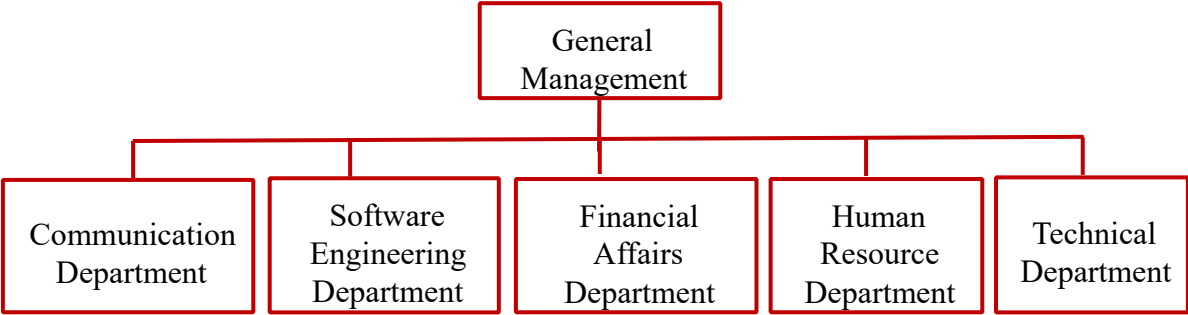


Figure 2. Functional Organization of REALIZE

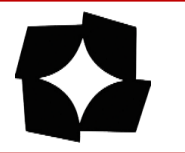


**IV. HARDWARE AND SOFTWARE RESOURCES OF THE
COMPANY**

1. Hardware Resources

N°	Designation	Quantity	Characteristics	Observation
1	MacBook Pro	1	APPLE	Very Good
2	Modem	1	CAMTEL	Very Good
3	LCD Monitor	1	HP	Excellent
4	Training Equipment and office Furniture	/	/	/

Table 1. *Hardware Resources of REALIZE*



2. Software Resources

As an enterprise specialized in IT, REALIZE also comprises of software resources such as:

Table 2. *Software Resources of REALIZE*

Designation	Software
Operation system	Windows 10, Mac OS, parrot Linux
Design tools	Photoshop
Integrated development environment (IDE)	Visual studio code,
Text editor	Sublime text, notepad++
Data base management system (DBMS)	MongoDB, PostgreSQL, MySQL
Web browser	Google chrome, Microsoft edge
Document editor	Microsoft office word
Presentation	Microsoft office PowerPoint



V. BRIEF PRESENTATION OF THE PROJECT THEME

Following our arrival at REALIZE, we were encouraged to suggest innovative project ideas for our internship. Following the mission of REALIZE that demonstrate the company's willingness to take active part in the sustainable development of the world through innovative solutions, we saw the room for creativity in our project idea and proposed the theme "**AI DRIVEN DECISION SUPPORT SYSTEM FOR INTELLIGENT DONOR MATCHING IN BLOOD DONATION NETWORKS**".



CONCLUSION

The insertion phase was a very educative one, we got to know more about the history of the company, how it started till it reached the level it is right now, how it is structured. I got to know the staff and other interns, in fact we felt at home. Finally, I decided to work on the theme **“AI DRIVEN DECISION SUPPORT SYSTEM FOR INTELLIGENT DONOR MATCHING IN BLOOD DONATION NETWORKS”**. Now that we are set with the theme we will work on, we can dive into the existing system phase in which we will study the existing systems this project already has so we can identify its limitations and formulate a problematic.



PART TWO: THE TECHNICAL PHASE



Preamble

This phase will explain how we analysed existing systems, worked up solutions and designed our own application. Every detail of the application is included in this phase.

Overview

FILE I. EXISTING SYSTEMS

FILE II. THE SPECIFICATION BOOK

FILE III. THE ANALYSIS PHASE

FILE IV. THE CONCEPTION PHASE

FILE V. THE DEPLOYMENT PHASE

FILE VI. THE FUNCTIONAL PHASE

FILE VII. THE INSTALLATION AND USER GUIDE



FILE I. EXISTING SYSTEMS



Preamble

In this file, we shall give a detailed explanation of how the existing systems function. That is, what the existing systems are all about and also to give more precision concerning the application we are to produce. This chapter will also describe the problematic that led to the implementation of the proposed solution

Overview

INTRODUCTION

- I. THEME PRESENTATION
- II. STUDY OF THE EXISTING SYSTEM
- III. CRITICISM OF THE EXISTING SYSTEM
- IV. PROBLEMATIC
- V. PROPOSED SOLUTION

CONCLUSION



INTRODUCTION

As Engineers our role in the society is problem-solving which is targeted at making life easier for everyone and in order to bring forth a solution, we ought to ask ourselves some questions to know why the problem exist in the first place and how it is being dealt with at the time. The existing system party of our report is very important. Here, we shall present our theme to improve our understanding of it, increase the likelihood of succeeding and present the study of the existing system which will guide us on the approach to take. After studying the existing system, we shall present our different critics concerning it. We shall also give the problematic. That is, the different problems phased by the existing system and finally we shall propose a solution to solve these problems.



I. THEME PRESENTATION

A theme is a term used to define something. It helps to delimitate a scope of work. In our case, we shall work on the theme “AI-DRIVEN DECISION-SUPPORT SYSTEM FOR INTELLIGENT DONOR MATCHING IN BLOOD DONATION NETWORKS” with prospective tenants in Yaoundé as case study. As this theme implies, we shall produce a platform that focuses mostly on property acquisition in Yaoundé and will help the prospective tenants in many ways to ease their search.

II. STUDY OF THE EXISTING SYSTEM

In Cameroon, blood supply is divided between voluntary donation, family replacement donation, and paid donation. Recent studies show that around 97% of donations come from replacement donors, family members who give blood in response to a relative's need. While only about 3% come from voluntary non-remunerated donors. This pattern holds across the country's blood banks; a 2021 review similarly found that of the pints collected that year, 90% came from family replacement donors. Hospital-based blood banks remain the norm, and most facilities require patients to bring at least two replacement donors and charge a fee per pint to cover collection and screening costs, making the system less a service and more a scramble.

Also, finding an eligible donor in time can be stressful when you are, for example:

- A patient's relative with a rare or mismatched blood type in your immediate circle.
- New to the city with few contacts to call on for help.
- Facing an emergency outside normal blood bank hours.

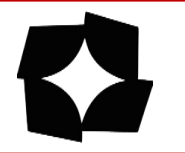
The existing system for blood donor mobilization in Cameroon is largely fragmented. Many hospitals and families rely on WhatsApp groups, Facebook appeals and word-of-mouth referrals to locate a compatible donor in time, with no central, verified record of who is actually eligible and available.

Some platforms already exist for this domain, some examples are;



Name	Description	Link
Infiuss Health	It is an online emergency blood supply platform which maintains a database of hospital's blood banks where patients can request a blood type.	https://infiuss.org/
Dashblood	A Cameroon-focused app that lets patients, families and hospitals create and share urgent blood requests to coordinate faster with willing donors.	https://mwm.ai/apps/dashblood/6772212650

Table 3. Existing Systems



III. CRITICISM OF THE EXISTING SYSTEM

After a meticulous study of the functioning of the above applications, we discovered that each of them comprises of both advantages and limitations. The following tables present the propose solutions in the scope of our project;

LIMITATION	CONSEQUENCE	SOLUTION
Requests are spread across Facebook groups, WhatsApp statuses and word of mouth.	Spend too much time scrolling from one DM to another and maybe finally not find a willing or compatible donor.	Introduction of a search feature where compatible donors are found and easily notified.
Operates only with partner hospitals and donors who previously donated	When partners are low on supply, they have nothing to provide and patients suffer from the shortage.	By connection directly to individual donors rather than depending only on hospital stock.
Charges a request but with no response guarantee. Smartphone/QR familiarity is required to navigate.	Users with financial constraints are not able to afford to submit a request. The platform maybe confusing to the less intellectual.	Donation requests are free of charge. The platform is easy to navigate and use.
No pre-screening before donor confirmation.	Matched donors may end up wasting both their time and the patient's by getting to the hospital and finding out they're ineligible to donate at that time.	Introducing a pre-screening feature with AI in order to minimize time wasted and make the process easier and smoother.

Table 4. Criticism of Existing Systems



IV. PROBLEMATIC

The study of the functioning of the existing system has permitted us to identify certain critics. As a result of those different critics, the major problem is; **how can donor mobilization for blood transfusion in Cameroon move from a broadcast-and-hope model to an intelligent, predictive system that identifies and prioritizes the right donors, at the right time, before a shortage becomes a life-threatening delay?**

V. PROPOSED SOLUTIONS

In order to solve the problems listed above, we have to develop an application based on the following points;

Blood Donation Awareness	• Hospitals can submit blood donation requests • Hospitals can create and publish campaigns. • Donors can view requests and accept.
Time Saving	• An AI model ranks compatible donors. • Matched donors are questioned by an AI model to ensure their eligibility • Notifications as well as messages are sent to the matched donors so they're aware of the request.
Communication	• Hospitals can communicate with matched donors on the platform



CONCLUSION

The theme constitutes a fundamental aspect of this project, as it defines the central focus and provides clear boundaries for the scope of work. It serves as a guide for identifying the system under study and determining the objectives to be achieved. Moreover, the study of the existing system remains an indispensable phase, as it enables a comprehensive understanding of its current structure and operations. In order to enhance a system effectively, it is first necessary to conduct a thorough analysis, identify existing limitations, and critically assess the challenges encountered. Upon completion of this analysis, the underlying problems can be clearly outlined, and appropriate solutions can be proposed to address them. This methodological approach not only facilitates the correction of identified weaknesses but also contributes to the overall improvement, efficiency, and sustainability of the proposed system.



FILE II: SPECIFICATION BOOK



Preamble

The specification book outlines the goals to be achieved through this project and the responsibilities of the different parties involved in the project. It specifies and describes the subject and the needs of the users, as well as the conditions necessary to realize the project

Overview

INTRODUCTION

- I. CONTEXT AND JUSTIFICATION
- II. OBJECTIVES OF THE PROJECT
- III. EXPRESSION OF NEEDS

CONCLUSION



INTRODUCTION

The specification book of our reports helps us provide details about our theme, to improve our understanding of it and increase the likelihood of it succeeding. To delimitate the scope of our project, we will specify the context of our theme. From the context, we will list the problems we have identified in our context and that we have decided to address throughout the project. After presenting the solution we will talk about the objective, we have set for ourselves and for the project. Also, we will explore the needs to which our system will respond both at the functional and non-functional level. We will then establish a plan which we will follow to complete our project on time and look at the estimated financial requirement for our project. From here, we will discuss what is expected of us by the end of the project under the project deliverables



I. CONTEXT AND JUSTIFICATION

"A life saved by a timely transfusion is a life given a second chance — but only if the blood arrives in time." By Asaah Andu.

In Cameroon, and particularly in hospitals across cities like Yaounde and Douala, patients in urgent need of blood transfusions depend on a system that is dangerously slow and disorganized. Blood banks frequently run dry, and when they do, the responsibility to "find blood" falls back on the patient's own family and friends, often within a matter of hours.

For instance, imagine a young man rushed to a hospital in Yaounde after a road accident, bleeding heavily and in urgent need of a transfusion. The doctor checks the blood bank and finds it empty. His family is told to find a donor themselves. They begin calling relatives, posting in WhatsApp groups, and reaching out to old donor contacts, hoping someone with the right blood type responds in time. Some contacts are ineligible to donate that day. Others are willing but live far away. The hospital sends out a generic SMS blast to everyone on an old donor list, but most never reply; some feel over-contacted and simply ignore it. Hours pass while the family waits, unsure if help will arrive before it's too late.

This is not an isolated case. Thousands of patients across the country face the same barrier every day: no way to reach eligible, nearby donors quickly, no way to know who is actually available before contacting them, and no way to mobilize help without wearing out the same tired pool of donors.

So the pressing questions arise:

- How can hospitals identify and reach the right eligible donors at the moment of need?
- How can donor mobilization prioritize proximity and urgency, rather than relying on chance and word of mouth?
- How can we reduce donor fatigue and wasted time so that willing donors keep responding when it matters most?

These challenges form the basis of the theme **"AI-Driven Decision-Support System for Intelligent Donor Matching in Blood Donation Networks"** with hospitals and blood banks in Yaoundé as the case study.



II. OBJECTIVES OF THE PROJECT

1. General Objective

To design and develop an AI-powered blood donation notification platform (BloodBridge) where hospitals can submit donation requests in order to find compatible donors. This improves the speed, accuracy, and efficiency of donor mobilization for hospitals and blood banks in Yaoundé.

2. Specific Objectives

- **To design** a system that allows hospitals and blood banks to submit verified blood requests directly, replacing informal, word-of-mouth appeals.
- **To develop** an AI model that predicts which registered donors are most likely to respond to a given request, based on factors such as donation history, eligibility, past responsiveness and ranks them depending on the request.
- **To implement** geospatial prioritization that ranks and notifies donors based on proximity to the requesting hospital, reducing response time.
- **To build** an eligibility pre-screening feature (via a conversational interface) that filters out ineligible donors before they are contacted, reducing wasted outreach.

III. EXPRESSION OF NEEDS

1. Functional needs

Functional needs describe what the users can do in the system. They are listed below;

Hospitals can submit donation requests

Donors can accept donation requests

2. Non-functional needs

a. Hardware requirements

- **Portability and ease of installation:** The hardware should be light-weight and easily transportable. It should require little or no skill to install and setup guide. For donors and hospital blood bank managers, the application should work seamlessly on common devices (smartphones, tablets and laptops).



b. Software requirements

- **Maintenance and Reusability:** The system should be built on a modular and scalable architecture to simplify updates, bug fixes, and the integration of future features such as payment systems, verification services, or AI-powered recommendations.
- **Efficiency:** The solution must effectively address key problems in the existing housing system such as exploitation by middlemen, hidden costs, fraud, and unreliable access to property information.
- **Speed:** To ensure smooth navigation, the platform must load quickly and handle multiple concurrent users without slowing down, especially during peak search periods (e.g., academic semesters when student housing demand spikes).
- **Security:** Since the system may manage sensitive data (user profiles, property ownership documents, and possibly financial transactions), strong encryption and secure authentication (multi-factor login, data encryption, and secure storage) must be implemented to guarantee authorized access only.
- **Ease of Use:** The system must offer an intuitive and user-friendly interface so that potential donors and hospital blood bank workers can use the platform without requiring advanced technical knowledge. Navigation should be clear, with simple workflows for searching, posting, and contacting.



IV. PROJECT PLANNING

1. Chronogram of activities

Phase	Objective	Output	Duration	Period
Insertion	Collection of information on the enterprise	Insertion book	2 weeks	1 st - 14 th July
Existing System	Study of the existing system	Existing System	1 week	15 st – 21 st July
Specification Book	Specification of the user needs	Specification Book	1 week	22 nd – 26 th July
Analysis	Capture of needs Use Case and textual description modelling	Analysis Book	2 weeks	27 th - 9 th August
Conception	Preliminary conception and Detailed conception	Conception Book	2 weeks	10 th – 23 rd August
Realization	Implementation Unitary test Integration test Development, Deployment, Component diagrams	Realization Book	3 weeks	24 th August to 17 th September
Test of Functionalities	Testing of the software and debugging	Test of functionalities	1 week	18 th – 23 rd September
Installation and User Guide	Documenting software	User Guide	1 week	24 - 30 September

Table 5. *Chronogram of activities*

2. GANTT chart

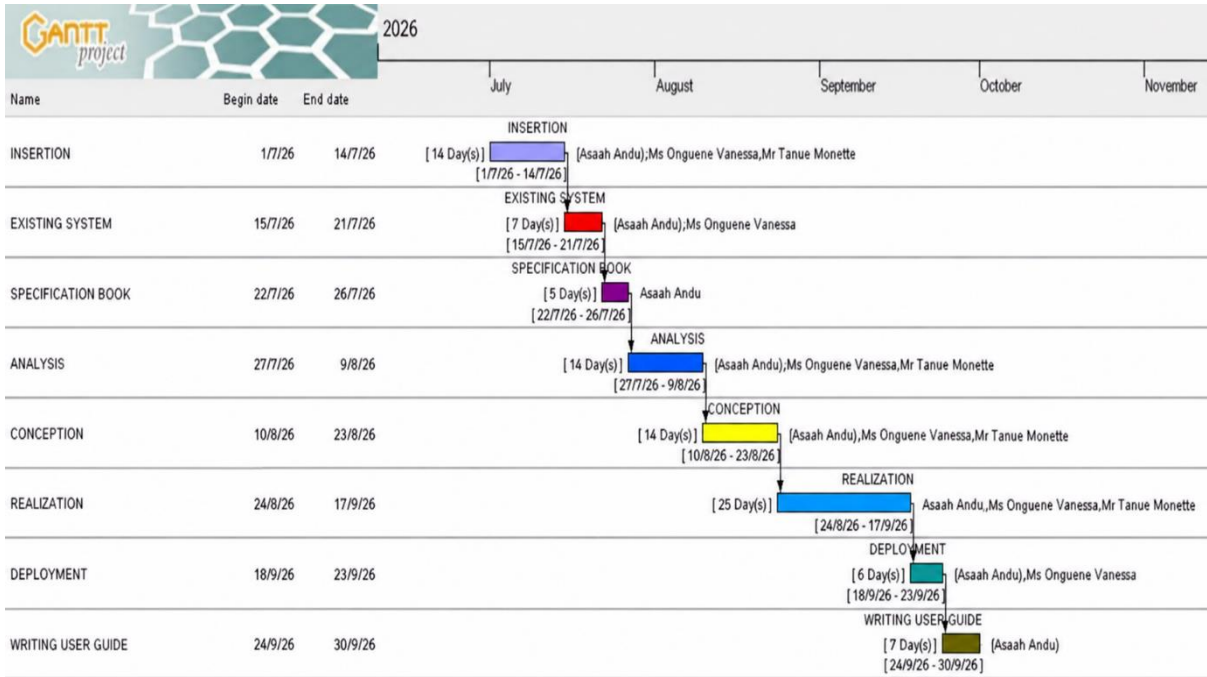


Figure 3. Gantt chart



V. ESTIMATED COST OF THE PROJECT

The cost of an application is not given randomly; numerous factors are to be taken into consideration. To determine the cost of our application, we shall consider the following resources;

1. Hardware Resources

Resources	Hardware	Usage	Quantity	Unit Cost (FCFA)
Computer	MSI Windows 11 Home, 12Gb RAM, DD 1.2TB hard disk	Main resource for the accomplishment of this project ; report writing, designs, analysis, coding	1	350,000
Removable Disk	32GB USB Key	For file transfer from one computer to another	1	8,000
Modem	4g CAMTEL WI-FI	Used to share internet connection amongst devices	1	20,000
Mobile Phone	IPhone 6Gb RAM, 256Gb hard disk	Testing of functionalities	1	140,000
TOTAL				518,000

Table 6. *Hardware resources of the project*



2. Software Resources

To realize our project, we needed the following softwares;

Resources	Software	Usage	Quantity	Unit Cost (FCFA)
Development tool	Visual studio code	Code editing	1	Freeware
Project planner	Gantt project	Project planning tool	1	Freeware
Operating system	Microsoft windows 11	Computer operating system	1	103,000
Text Editor	Microsoft office word 2016	For typing and formatting of report	1	161,000
Presentation	Microsoft office PowerPoint 2016	For making presentations	1	84,230
Modelling tool	Visual Paradigm Modeler version	Modelling the system in UML	1	3,350
Geo-location plan designing tool	Microsoft Visio 2016	To draw location plan	1	Free software
Database management system	MongoDB	Communing with the database	1	Free software
TOTAL				351,580

Table 7. *Software resources of the project*



3. Human Resources

For the realization of any project, humans play a very important role. To realize our project, we will need the following human resources;

Resources	Number	Cost Per Day (FCFA)	Number of Days	Cost (FCFA)
Project manager	1	30 000	60	1 800 000
Analyst	1	20 000	30	600 000
Designer	1	13 000	25	325 000
Programmer	1	10 000	30	300 000
TOTAL				3 025 000

Table 8. *Human resources of the project*

4. Total Project Estimated Cost

Resource	Price (FCFA)	Component Cost (FCFA)
Hardware resource	828 000	4 204 580
Software resource	351 580	
Human resource	3 025 000	
Charges		
Charges (5% Total resources)	210, 229	630 687
Unforeseen (10% Total resources)	420, 458	
Total project cost		4 835 267

Table 9. *Total project estimated cost*

Drawn at the sum of: Four Million Eight Hundred and Thirty-five Thousand Two Hundred and Sixty-seven Francs CFA

VI. PROJECT CONSTRAINTS

In our scope of work, we encountered numbers of problems which include:

- Cost constraints: The cost must be respected in order to provide quality work
- Integrity constraints: The application must be well secured
- Delay constraints: The realization of the project must fulfil the time constraints requested and meet with the set objectives
- Quality Constrains: After setting a price and delay, we are required to produce a reusable, flexible and evolutionary application

1. List of participants

Table 10. *List of Participants*

Name	Function	Role
Mr. TANUE Monette	Follows up interns at the company level	Professional Supervisor
Mme ONGUENE VANESSA	Follows up student at the academic level	Academic Supervisor
ASAAH ANDU ONYA	REALIZE Student Intern	Student at AICS

2. Project Deliverables

In project management, any component materializing the result of a realization service is called a deliverable. In the case of our project, the deliverables are: A report composed of the following document. We are supposed to submit the following at the end of this project:

- The application
- The User guides
- The power point



CONCLUSION

The specification book helps us to site the different needs we have in order to implement our application to ease the ability for hospital blood bank managers to find compatible and eligible donors. With the time allocated for this section, we were able to accomplish different task. As such, we shall dive directly into the next phase which is the analysis phase.



FILE III: ANALYSIS FILE



Preamble

The analysis phase which follows the specification book, represents a detailed analysis of the problem, the solution, the analysis method chosen and the reason of choice.

Overview

INTRODUCTION

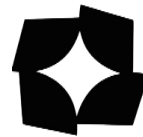
- I. PRESENTATION OF THE ANALYSIS METHOD
 - i. Comparative Study of UML and MERISE
 - ii. Comparative Study of the Unified Process
- II. MODELLING
 - i. Use case diagram
 - ii. Communication diagram
 - iii. Sequence diagram
 - iv. Activity diagram

CONCLUSION



INTRODUCTION

The analysis book permits us to examine in an explicit way the existing system, its limitations and how we can remedy them. We will also describe in details the modelling language known as UML (Unified Modelling Language) which is coupled with Two Track Unified Process (2TUP) to form a method and its justification why we decided to use it in preference of another. Then we will dive directly into the modelling of the proposed solution consisting of diagrams that meets the requirements of the functional needs.



I. PRESENTATION OF THE ANALYSIS METHOD

1. Comparative Study of UML and MERISE

a. Modelling with MERISE

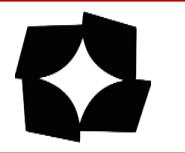
MERISE stands for “*Méthode d’Etude et de Réalisation Informatique pour des Systèmes d’Entreprise*”. Although it is prescriptive to some extent, MERISE permits the participation of end users and senior management as well as data processing professionals in its decision cycle. MERISE is a method for designing, developing and carrying out IT projects. The goal of this method is to achieve the design of an information system. The MERISE method is based on the separation of data and processing to be carried out in several conceptual and physical models. The essentials of the approach lie in its three cycles: the decision cycle, the life cycle and the abstraction cycle, which cover data and process elements equally. The separation of data and processing ensures longevity in model. Indeed, the arrangement of data does not have to be often overhauled, while treatments are more frequently

b. Modelling with UML

The unified modelling language (UML) is a general purpose, developmental modelling language in the field of software engineering that is intended to provide a standard way to visualize the design of a system. The Unified Modelling Language (UML) was standardized in January 1997 by the Object Management Group (OMG) which is an American association created in 1989 and aims to promote and standardize the object model in all its forms. In 2005, UML since 2015 is in its version 2.5. This version consists of fourteen diagrams classified into structural and behavioural diagrams.

Table 11. *Structural and Behavioural Diagrams*

Structural Diagrams	Behavioural Diagrams
Class diagram	Use case diagram
Object diagram	Activity diagram
Component diagram	State machine diagram
Deployment diagram	Sequence diagram
Package diagram	Communication diagram
Profile diagram	Global Interaction diagram



The following table shows the difference between UML and MERISE;

Table 12. *Difference between UML and MERISE*

UML	MERISE
Modelling language	Analysis method
Object oriented base	Procedural approach based
Association of data and processes	Separation of data and processes
It is not a method but an object-oriented modelling language which is associated to a method i.e., the 2TUP method	It is a method of analysis and conception of an information system i.e., it uses a systematic approach

2. Comparative study of the Unified Process

a. The Unified Process (UP)

A unified Process is a process of development of software constructed on UML; it is iterative, incremental, centred on architecture, driven by use cases and requirements.

- **Iteration** is distinct sequence of activities with a basic plan and evaluation criterion that produces an internal or external output. Either the content of an iteration is improved or the evolution of the system is evaluated by users.
- **An increment** is the difference between two released products at the end of two iterations. Each iteration that the group is capable of integrating the technical environment in order to develop a final product and give users the possibility of having tangible results.
- **Centred on architecture** the different models derived during the establishment of system must be reliable and coherent.
- **Driven by use case and requirements** enables the clear definition of a users' needs and priorities respectively thereby minimizing the risk of project failure.

b. The Two Track Unified Process (2TUP)

The 2TUP is a unified process which is built on UML and has as objective to bring solution to constraints of functional and technical changes imposed on information systems by strengthening controls on development capacities. It proposes a Y-shaped development life cycle that separates the functional aspect from the technical aspects, and the merging of these two forms the implementation aspect. 2TUP distinguishes therefore two branches: the functional and technical branches, the combination of the result of these two branches forms the third: the realization branch – where we realize our system.

The 2TUP method has numbers of advantages, some of the include;

- It enables system extension
- It enables the introduction of new modules
- It enables project analysis flexibility
- It eases comprehension and conception

The diagram bellow illustrates the branches of 2TUP;

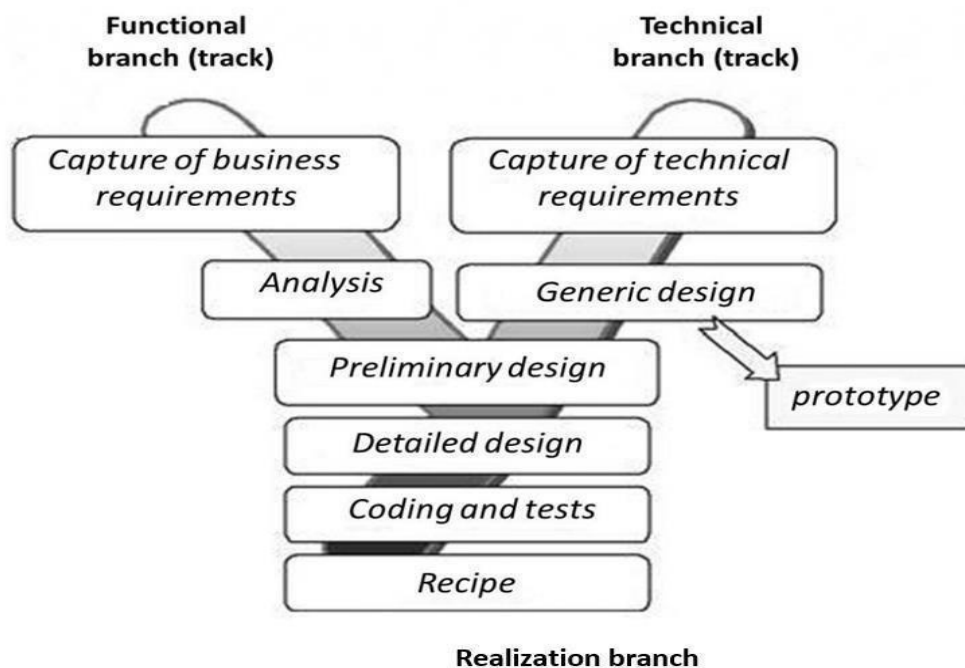


Figure 4. 2TUP Diagram



Let's describe the Y shape of the 2TUP method;

The Left branch (Functional branch)

It captures the functional needs of a system. This ensures the production of software that meets the needs/requirements of the user. The analysis here consists of studying precisely the functional specification in order to obtain an idea of what the system is going to realize and its result does not depend on any technology

The right branch (Technical branch)

The technical branch enumerates the technical needs and proposes a generic design validated by a prototype. The technical needs include constraints and choices related to the conception of the system, the tools and equipment as well as the integration constraint with the existing system condition. The different diagrams are shown in the tables below.

The middle branch (Realization or implementation branch)

In this branch, we study the preliminary conception, detailed conception, and documentation of the system. The realization branch supports the following:

- **Preliminary conception:** This is the most sensitive step of 2TUP as it is the confluence of the functional and technical branch. It is completed when the deployment model, the operating model, the logical model, interphases and the software configuration model are defined.
- **Detailed conception:** This is the detailed design of each feature of the system.
- **Coding and testing:** This is the phase where we program the designed features and test the coded features.
- **The recipe:** Also known as the deliverables is the validation phase of the functions of the developed system.



3. Justification of Method of Analysis

As method of analysis, we chose the Unified Modelling Language associated with the 2TUP software development process / method. This choice was made as a result of the following criteria:

- UML is the current standard for programming in an object-oriented language. For this reason, it is widely understood and well known making it easy for a new programmer to join the project and be productive from the very first day.
- UML diagrams allow teams to virtualize how a project is or will be working, and they can be used in any field, not just software engineering. The diagrams will allow teams to virtualize together how a system or a process will work or did work. It can provide new ideas for how teams have to collaborate to achieve the goal of the workflow process.
- 2TUP is centred on the creation and maintenance of a model, rather than the production of mountain documents.
- 2TUP is user oriented as it permits the development of software that responds to the needs of the users through the study of the user needs.
- 2TUP is iterative and incremental; hence it enables the project team to produce refined amelioration if necessary and easily integrate it in the already existing system.

II. MODELLING OF PROPOSED SOLUTION

1. Use Case Diagram

a. Definition

The use case diagram shows the functionalities of the system, their interdependencies, and the relationship that exist between a use/actor and the system. A use case is a case (or situation) where your system is used to fulfil one or more user requirement. It captures a piece of functionality that a system performs. The main objective of the use case diagram is to; identify the function of the system and provide high level view of the system. Generally, there exist two types of use case description; they are;

- The textual description of each use cases;
- The use case diagram

A use case generally comprises of a name, a brief description of its flow, the context in which it is used, the related actor and finally a detailed description.

b. Formalism

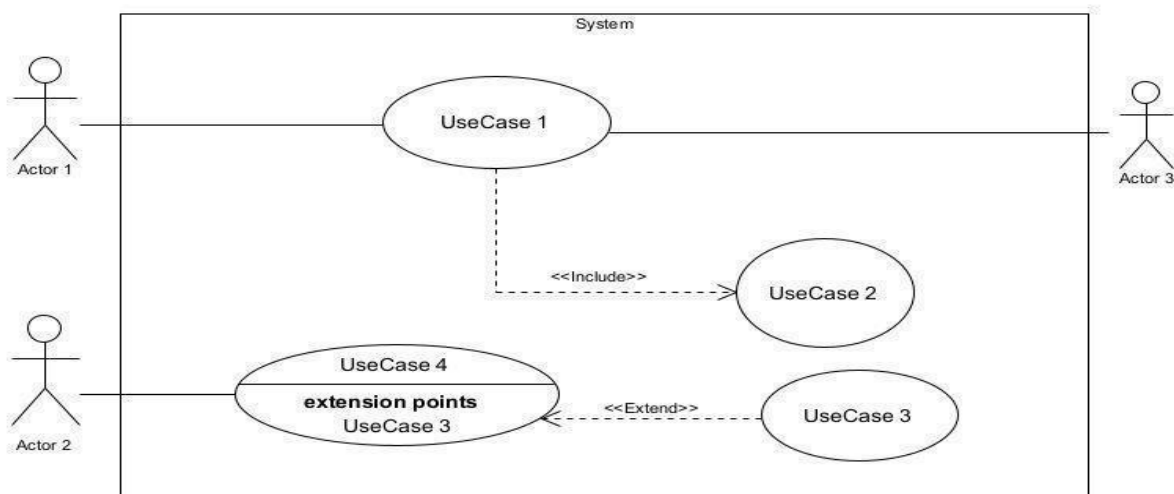
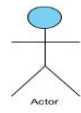
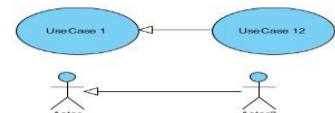
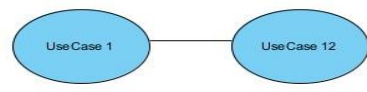
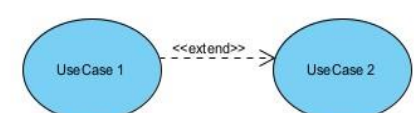
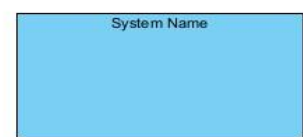
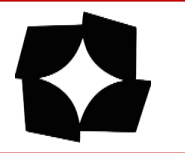


Figure 5. Use case diagram formalism

The following table presents the formalism of a use case diagram

Table 13. *Formalism of use case Diagram*

Element	Description	Notation
Actor	It refers to the role played by the person or something when interacting with the system	
Use case	It represents a situation where the system is used to fulfil one or more user requirement	
Generalization	It indicates that a child use case/ actor is a specialization of a parent use case/actor	
Association	It indicates that there is a relationship existing between two or more use case	
Include	It indicates that the use case at the head of the dotted arrow completely reuses all the steps from the use case being included	
Extend	It indicates that the use case at the head of the dotted arrow optionally reuses all the steps from the use case included	
System boundaries	It represents the system and shows a clear separation between the actors and the use cases	



c. Identification of system actors

On the basis of the proceeding research performed, we identified the following actors;

Table 14. *Identification of actors and their use cases*

Actors	Use Case
Donor	• Consult blood request • Complete eligibility screening • Find donation center
Abstract User	• Send message • Chat with AI • Edit profile
Hospital	• Submit blood request • Consult active request • Create campaign • Submit KYC request
Admin	• View user account • View KYC request

d. General Use Case Diagram

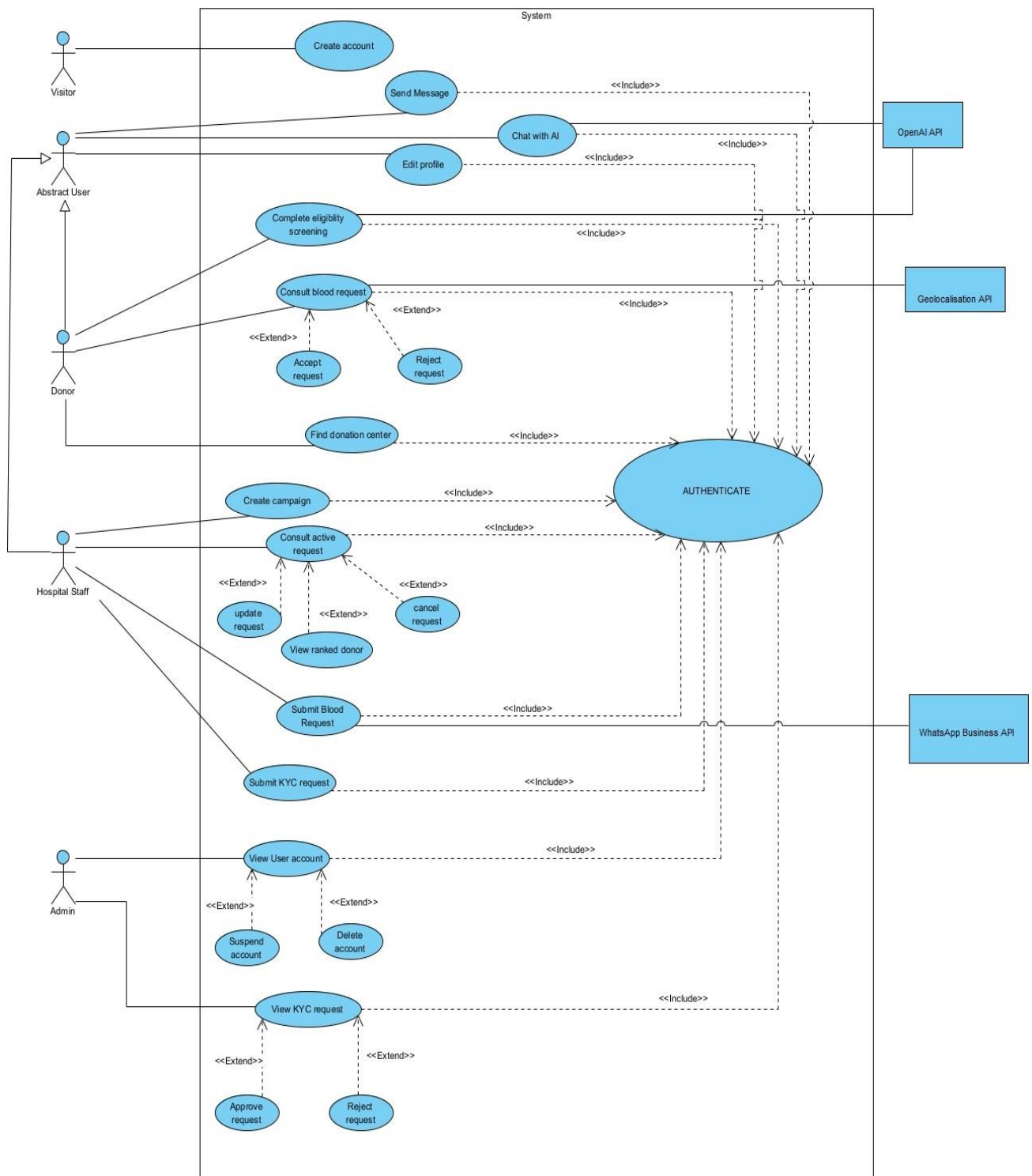


Figure 6. General Use Case Diagram

e. View Blood Request Use Case Diagram

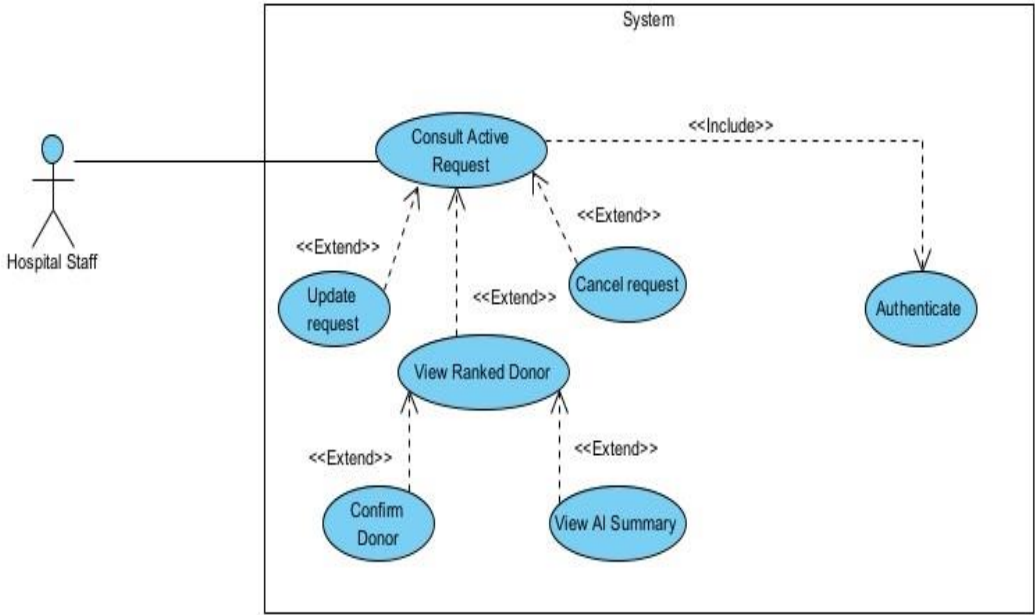


Figure 7. *Consult Active Request Use Case Diagram*



f. Textual description of use cases

Every use case is associated with series of actions that represents the desired functionalities and the strategies alternatively used in case of failure or if an error occurs. A use case is an abstraction of several execution paths. An instance of a use case is called a "scenario". The number of instances a use case can have, is infinite.

Use case description is done following a scenery which define the interactions that constitute that case. This description is done simply by using text that can have understood by the person in the application. The description usually indicates what the user can do in the system. The scenery can be organized as follows;

- **Nominal or principal scenario:** It refers to the principal instance of the use case. It is the scenario of success;
- **Secondary scenario:** It refers to an alternative case, exception case or an error;

The textual description is as follows:

- **The precondition:** It refers to the set of conditions that must be fulfilled before the beginning of a use case
- **The trigger:** It is the element that triggers the use case
- **The dialogue:** It refers to the nominal and secondary scenario
- **The post condition:** It is consisting of the different mechanisms that leads to the end of a use case process
- **The end:** It is the ending state of the use case.



The following tables present the textual description of some use cases of our system;

Title	Authentication
Summary	The user needs to authenticate
Actors	Donor, Hospital Staff, Administrator
Date	September 1, 2026
Stakeholder	BLOODBRIDGE directors
Version	1.0
Precondition (s)	The user does not have an account on the platform
Triggers	The user launches the application
Nominal Scenario	1. The system displays the login form 2. The user clicks on Sign in button 3. The system displays the Sign in form 4. The user fills and submits the form 5. The system verifies the conformity of the form 6. The system sends the data to the database in form of query 7. The database returns the result of the query 8. The system displays a success message to the user
Alternative Scenarios	1. In step 4 of the nominal scenario, the user enters mismatched or missing information. 2. The system displays an error message then return to step 3 of the nominal scenario.
Post-condition of success	The user has access to his/her dashboard
Post-condition of failure	The user does not have access to his/her dashboard
Non-functional requirement	The entry of the password must not be visible on the screen

Table 15. *Authentication textual description*

Title	Submit Blood Request
Summary	The Hospital Staff sends a Blood donation request.
Actors	Hospital Staff
Date	September 1, 2026
Stakeholder	BLOODBRIDGE directors
Version	1.0
Precondition (s)	The actor is authenticated
Triggers	The user clicks on New request button.
Nominal Scenario	1. The system displays a form to fill 2. The actor fills and submits the form 3. The system verifies conformity of the form. 4. The system sends the Blood Request query to the DBMS 5. The DBMS returns the result of the query. 6. The system displays a success message.
Alternative Scenarios	3. In step 3 of the nominal scenario, the user enters mismatched or missing information. 4. The system displays an error message then return to step 2 of the nominal scenario. 5. In step 6, the DBMS may be disconnected or encounter network issues. 6. The system displays an error message then return to step 2 of the nominal scenario.
Post-condition of success	The tenant has requested a visit.
Post-condition of failure	The tenant has not requested a visit.

Table 16. *Submit Blood Request Textual description*

2. Communication Diagram

a. Definition

A communication diagram models the interactions between objects in a sequence. It also shows the links that are needed between participants to pass an interaction's message. Moreover, it provides an intuitive way to show the links between participants that are required for the events that make up the interaction. A communication diagram is made up of three things namely;

- The participant;
- The communication links between those participants;
- The messages that can be passed along those communication links;

b. Formalism

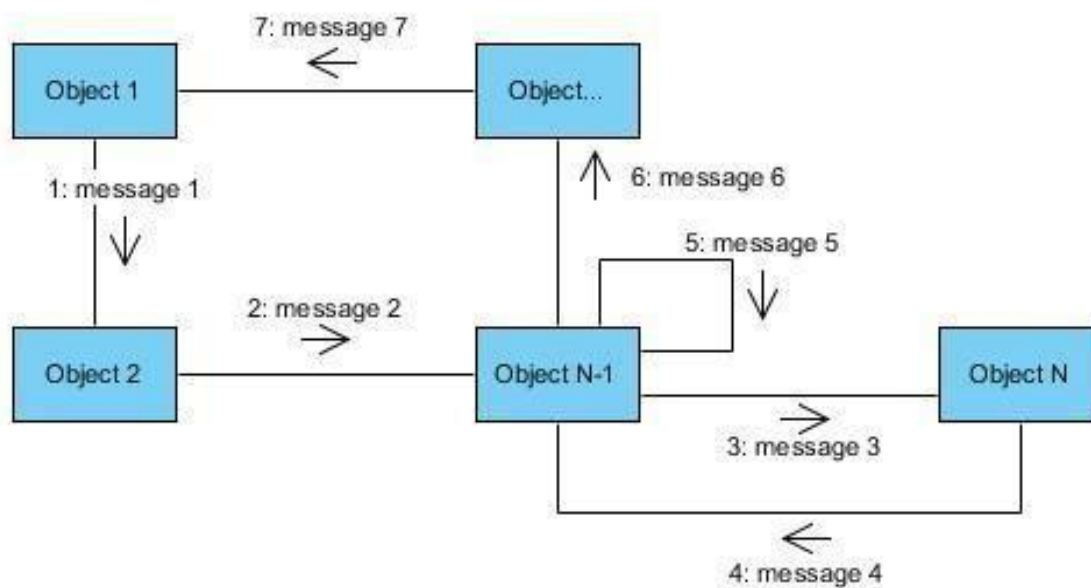
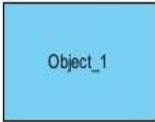
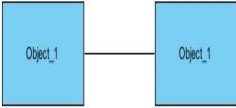
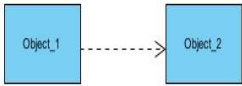


Figure 8. *Communication diagram formalism*

The following table presents the formalism of the communication diagram

Table 17. *Formalism of the communication diagram*

Element	Description	Notation
Participant/ Lifelines	It is represented by a rectangle with the participants name in the middle of it	
Communication Link	It is a link that allows messages to be passed between the different participants	
Message	Designs a particular communication between participants.	
Dependency	A dependency is a relationship that signifies a single or a set of model elements of their specification	

c. Authentication communication diagram

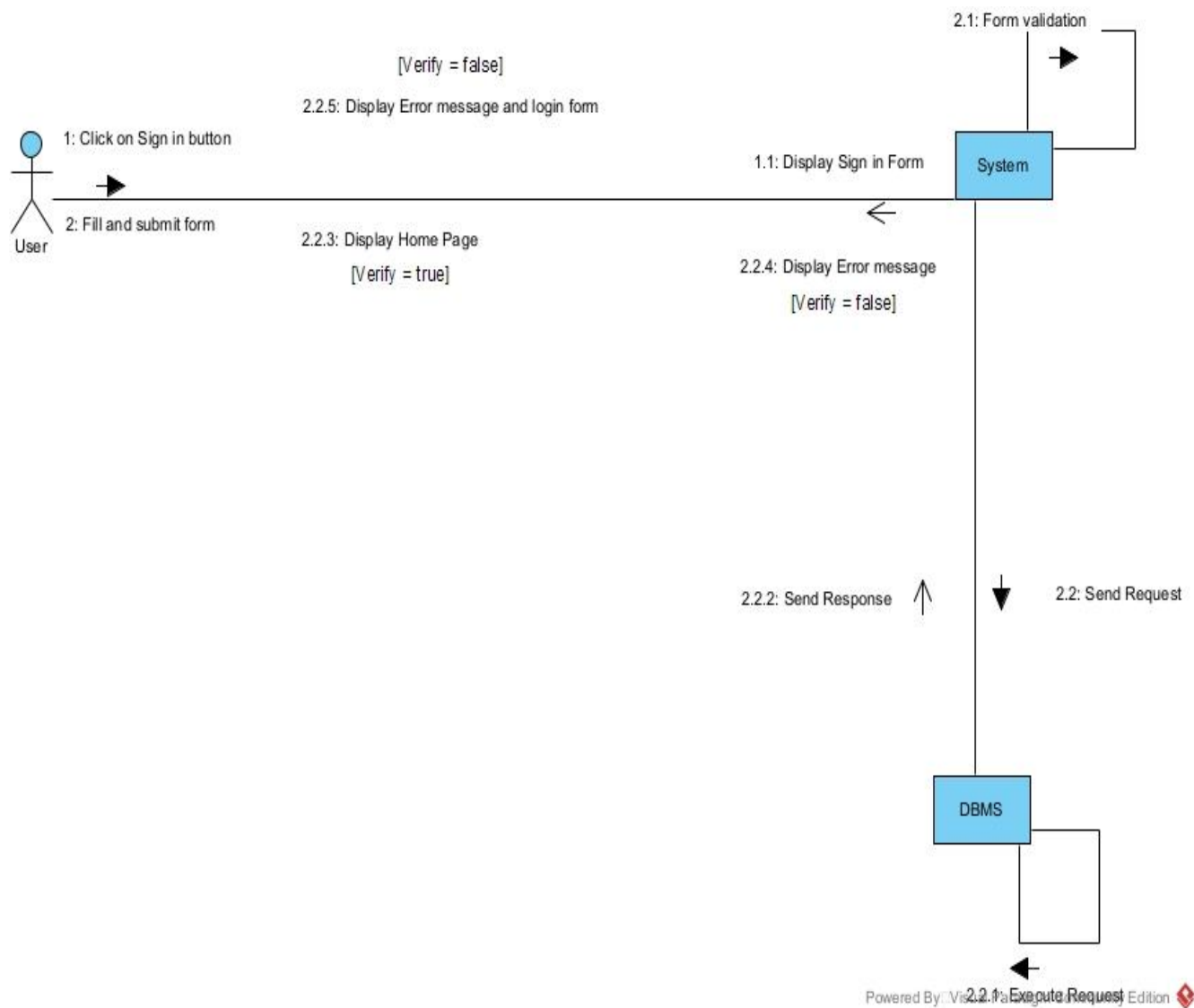


Figure 9. *Authentication communication diagram*

d. Blood Request Communication diagrams

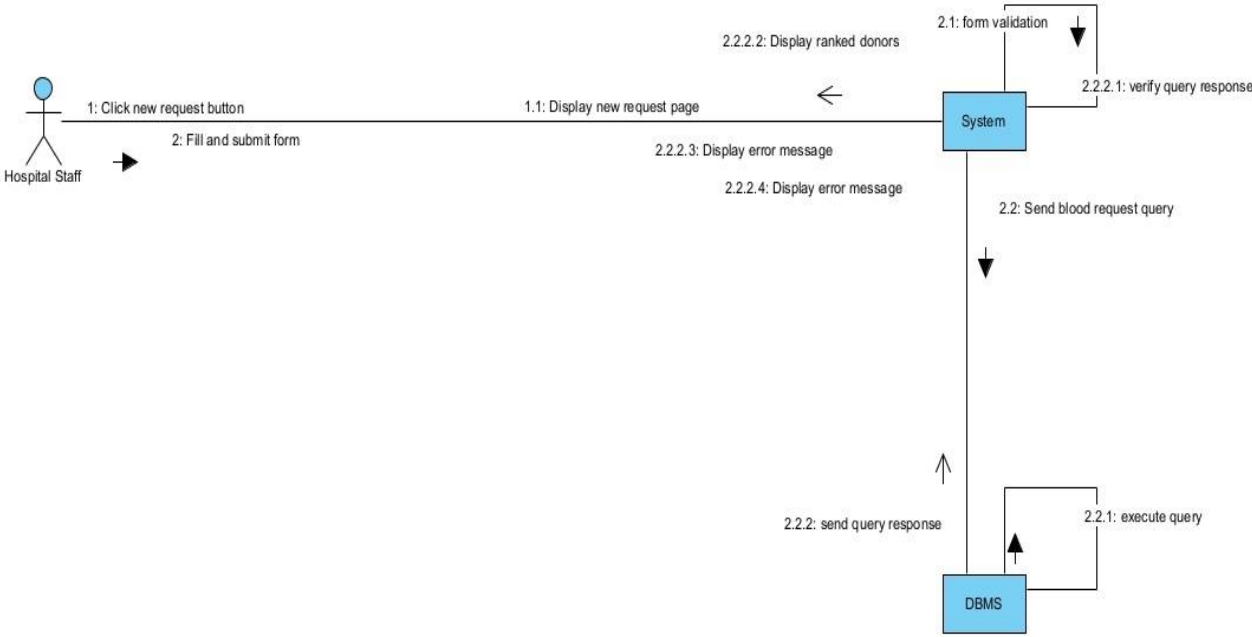


Figure 10. *Blood request communication diagram*

3. Sequence Diagram

a. Definition

A sequence diagram models the order of interaction between parts of our system. In other words, it describes interactions among classes in terms of an exchange of messages over time. Sequence diagrams are an important member of the group known as interaction diagrams. Using a sequence diagram, you can describe which interactions will be triggered when a particular use case is executed and in what order those interactions will occur.

b. Formalism

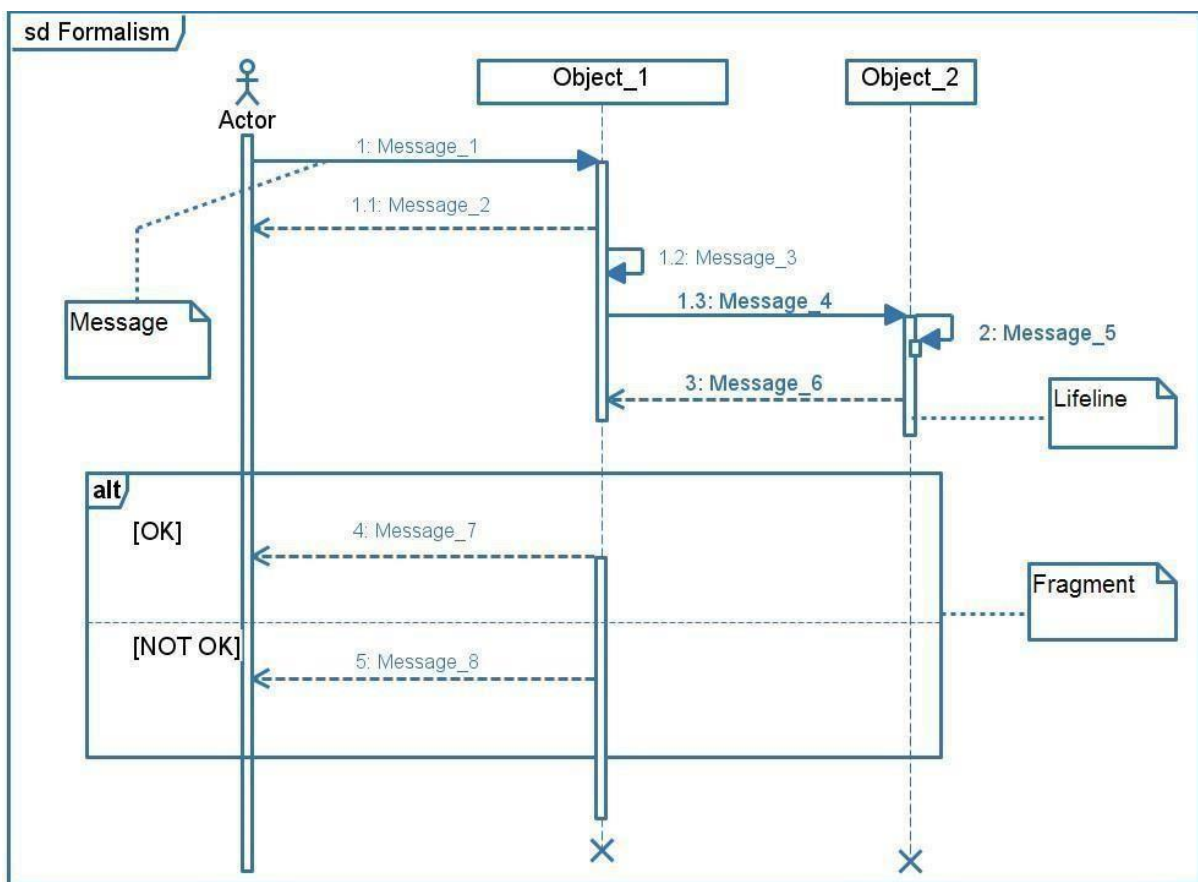
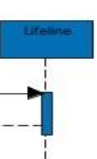
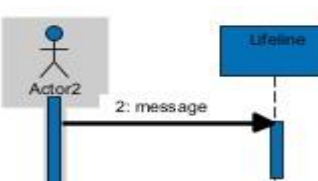
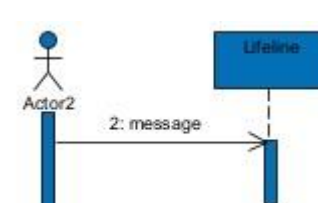
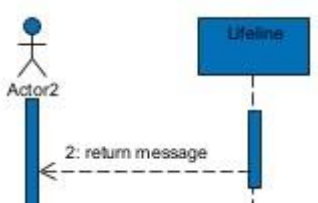


Figure 11. *Sequence diagram formalism*

The following table present the formalism of the sequence diagram

Table 18. *Formalism of sequence diagram*

Element	Description	Notation
Participants	They represent rows or objects instances that participate in the sequence being modelled. Each participant has a corresponding lifeline running down the page	
Activation bars	It indicates that a participant is active	
Synchronous message	It is involved when the message caller waits for the message receiver to return from the message invocation	
Asynchronous message	It is invoked when the message caller doesn't wait for the message invocation to return before carrying on through the rest of the interaction steps	
Return Message	It is an optional piece of notation that you can use at the end of an activation bar to show that the control flow of the activation returns to the participant that paused the original message	

c. Authentication sequence Diagram

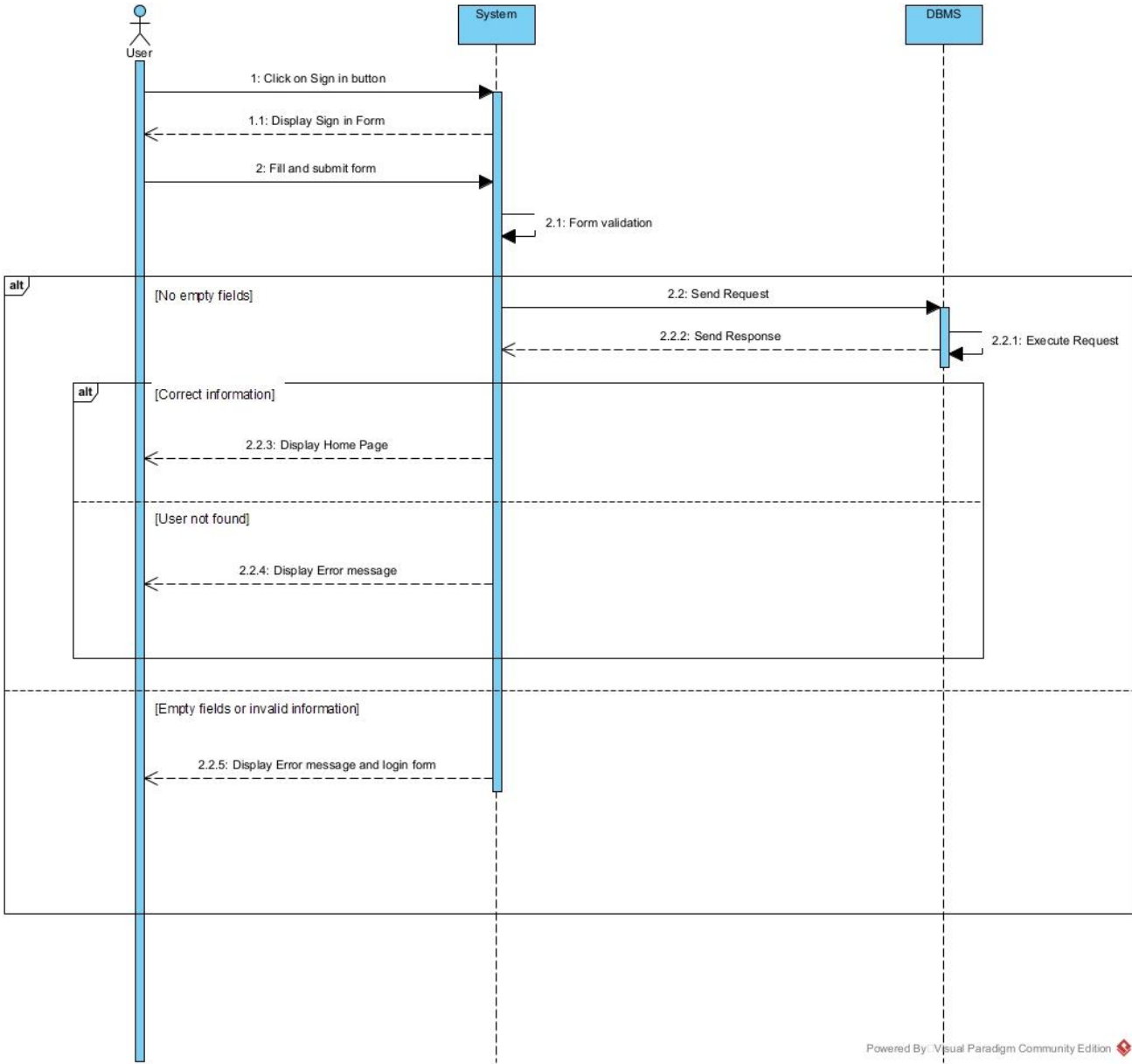


Figure 12. *Authentication sequence diagram*

d. Blood Request Sequence diagram

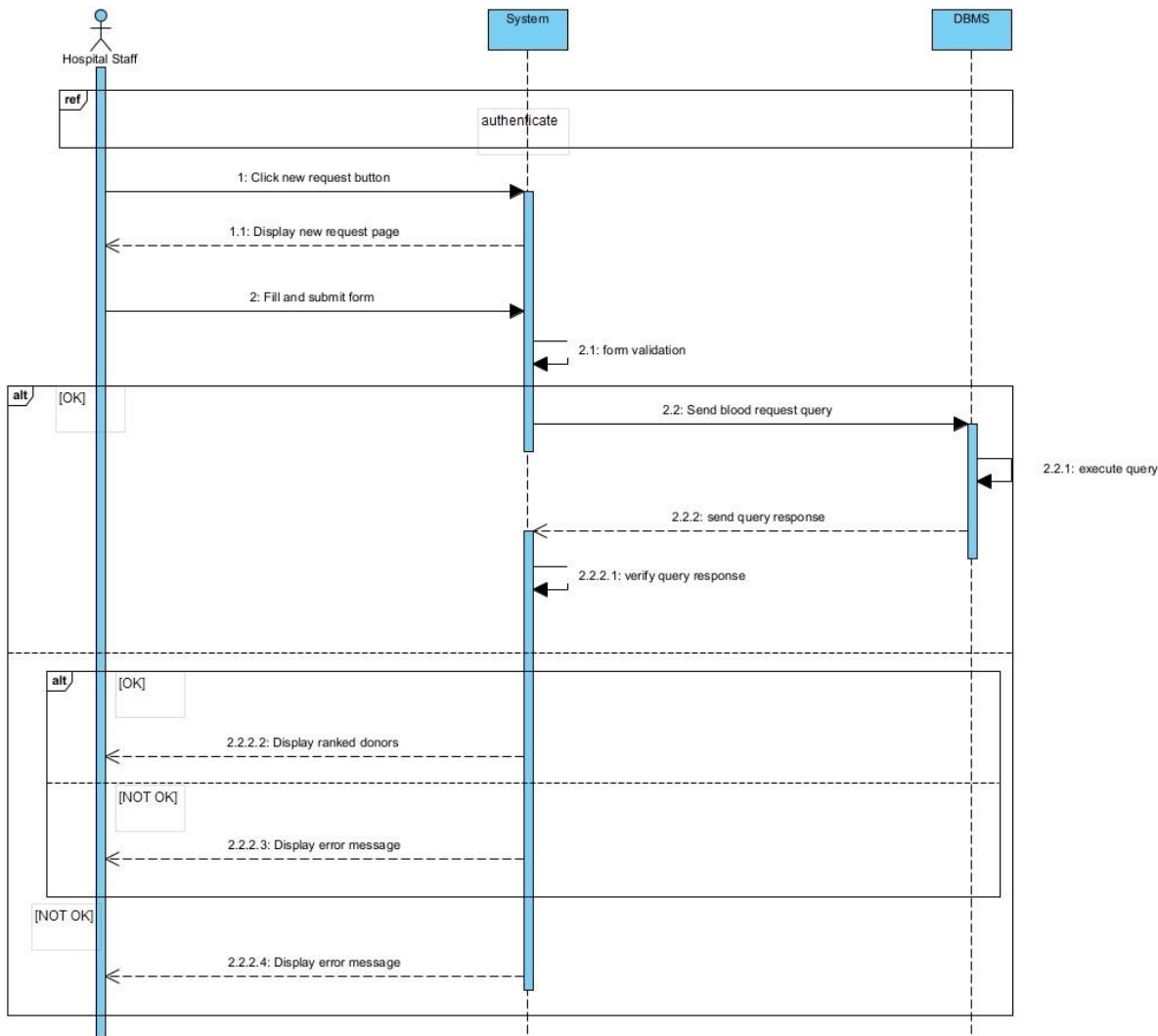


Figure 13. *Blood request sequence diagram*

4. Activity Diagram

a. Definition

An activity diagram is a graphical representation of workflows that show the steps needed in the realization of a process. Activity diagrams allow you to specify how your system will accomplish its goals. It shows a high-level action chained together to represent a process occurring in your system. Activity diagrams are one of the most accessible UML diagrams since they use symbols similar to the widely-known flowchart notation.

b. Formalism

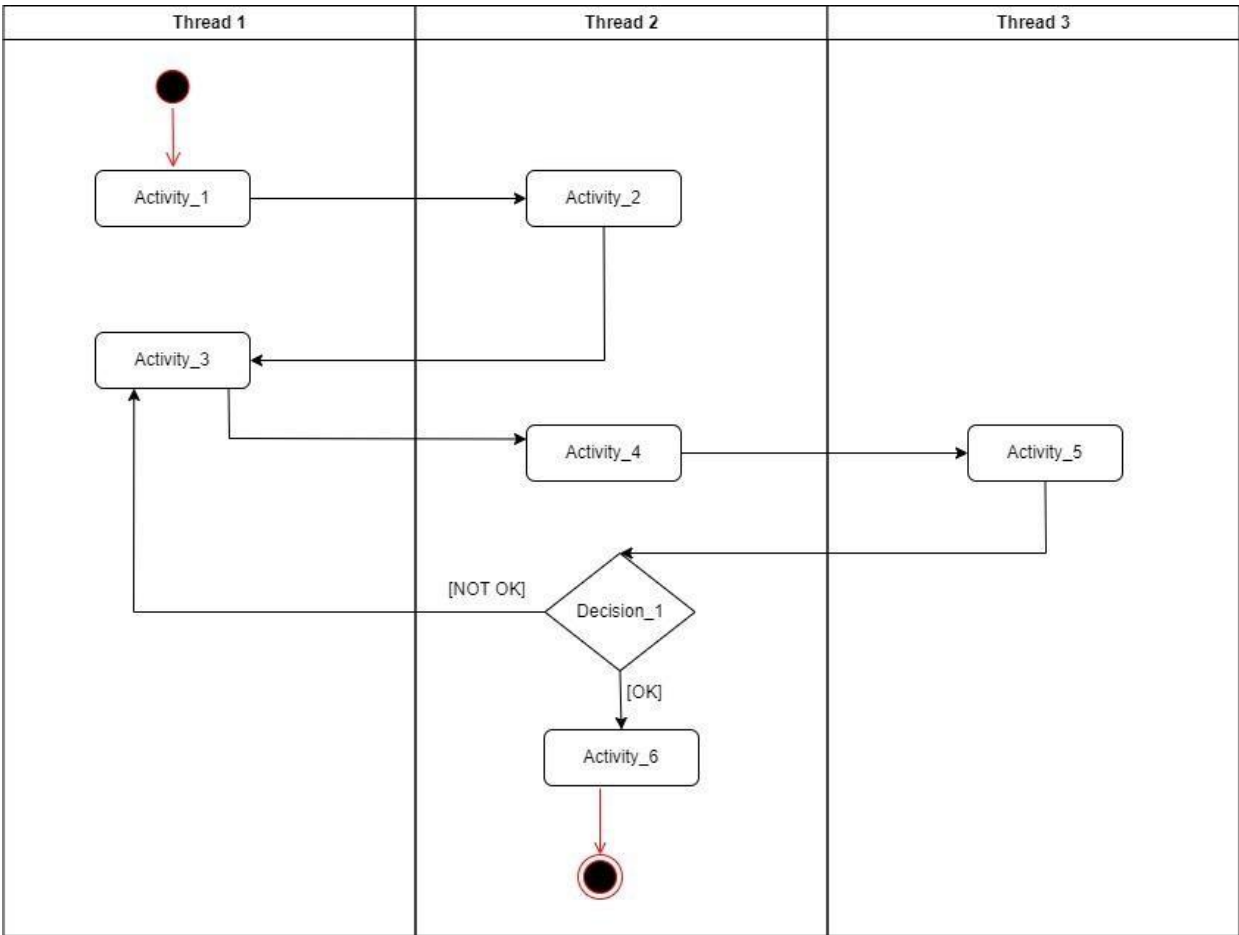
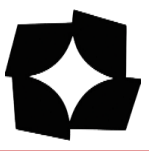


Figure 14. Activity diagram formalism



The following table represents the formalism of the activity diagram

Table 19. *Formalism of the activity diagram*

Element	Description	Notation
Activity	It describes an action performed by an object it is represented by a rounded rectangle	
Action	It can be de describe as a step in an activity	
Transition	When an activity state is complete, the next activity continuous with the process	
The corridor	All activities are place in a corridor which correspond to an actor. Objects are enumerated at the top of the column and vertical bars separate column to form a corridor	
Initial node	It shows the entry point of the first activity	
Final node	Stops all controls and object flows in an activity	
Decision node	Represents a test condition that slits an incoming activity edge into opposite outgoing activity edges	
Merge Node	It is used to reunite decision paths created using a decision node	

c. Authentication activity Diagram

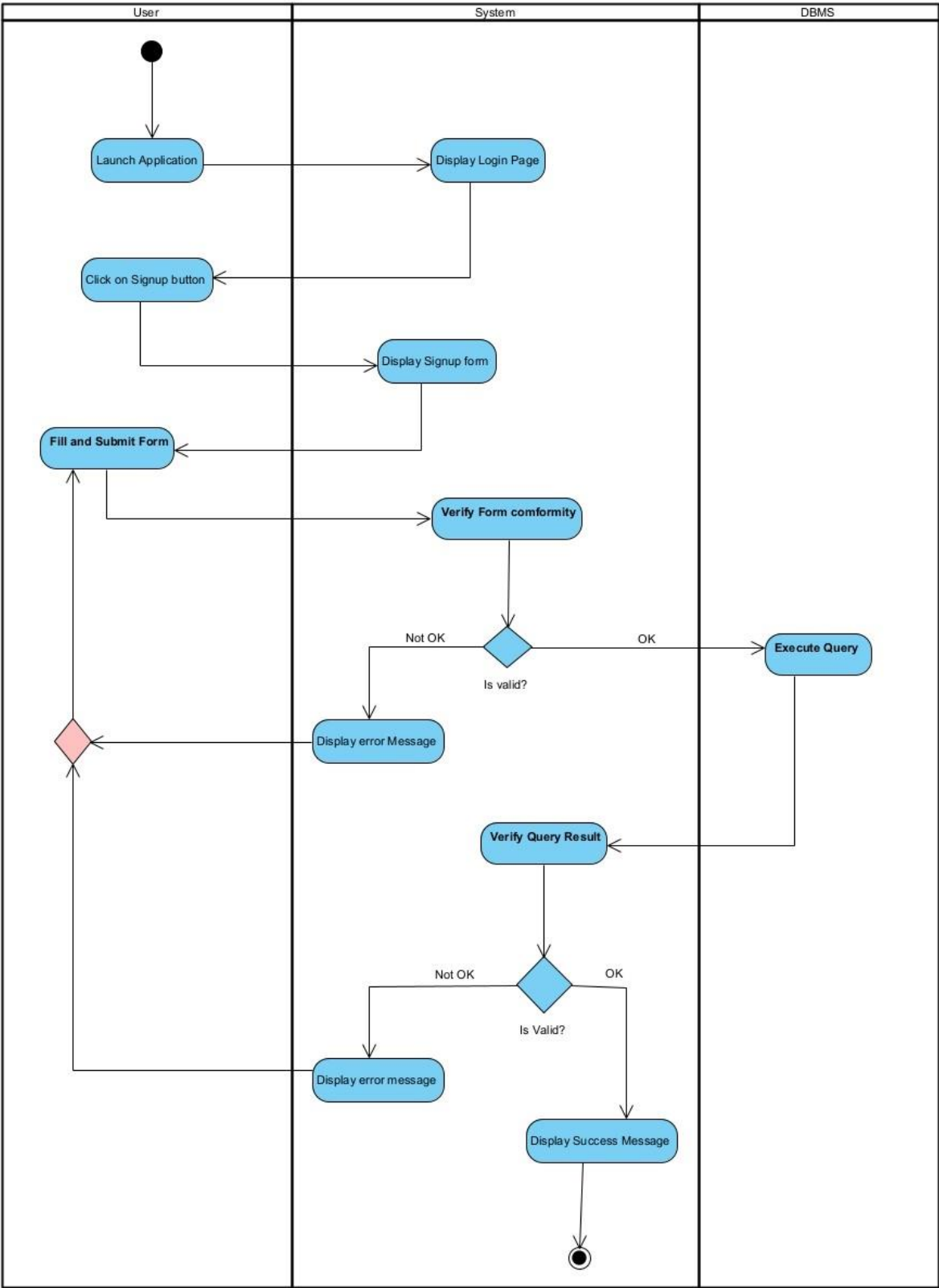


Figure 15. Authentication activity diagram

d. Submit Blood Request Activity Diagram

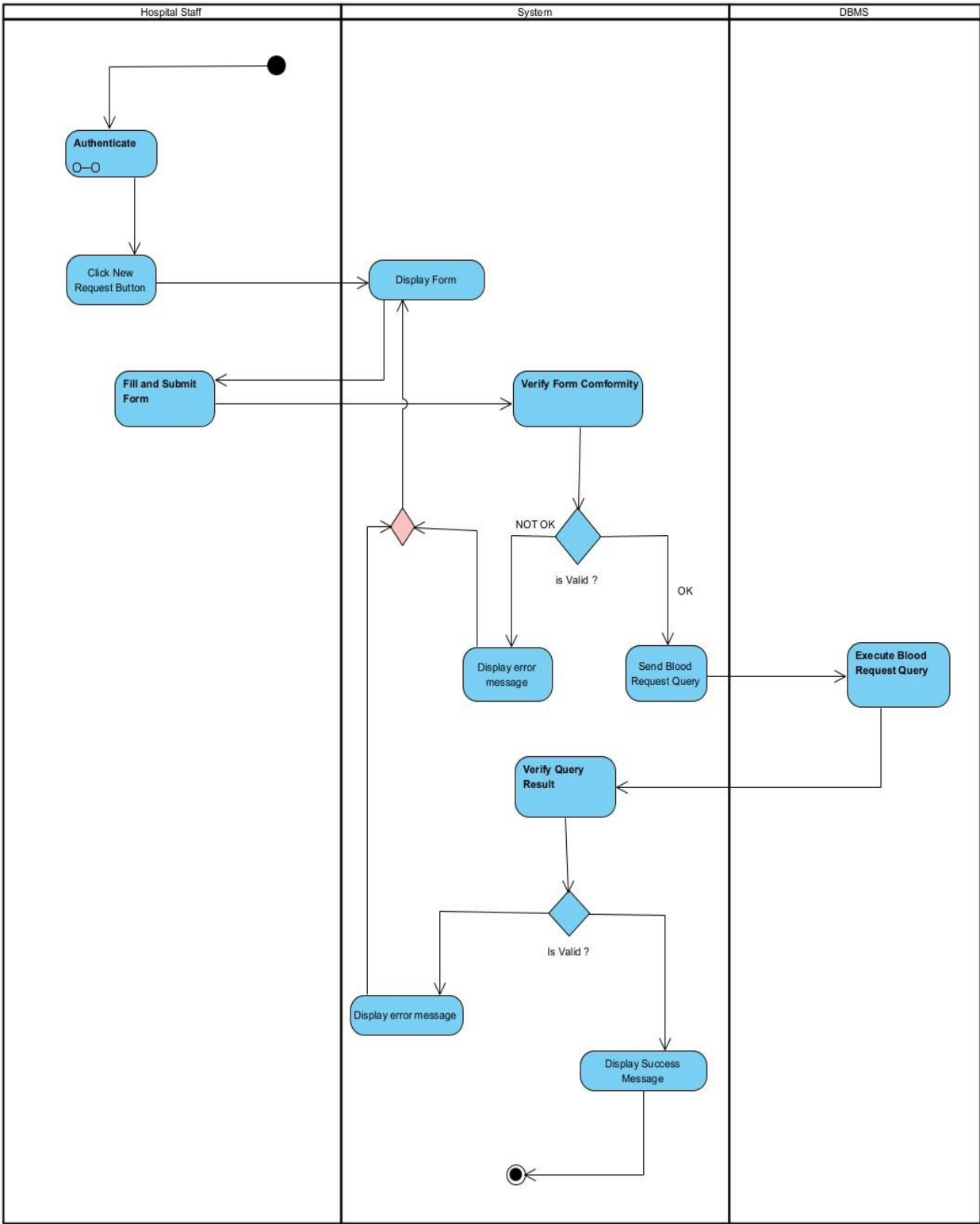


Figure 16. *Submit Blood Request Activity Diagram*



CONCLUSION

Arrived at the end of this part, where we choose the methodology of analysis for our project. We saw the use case diagrams that illustrate the relationship that exist between the system and users. We also saw the sequence and communication diagram which presented the interactions of part of the system with each other. Finally, we saw the activity diagram which presented the specifications on how the system will accomplish its goals. We shall now dive into the conception phase of our project.



FILE IV: CONCEPTION FILE



Preamble

The conception phase will permit to present a clean and ordered image of the future information system. It consists of static and dynamic structures. It helps us answer to the questions of how to conceive the problem? And are the technical needs to be used?

Overview

INTRODUCTION

- I. TECHNICAL BRANCH
 - GENERIC DESIGN
- II. CAPTURE OF TECHNICAL NEEDS
- III. RELATED UML DIAGRAMS
 - 1. Class Diagram
 - 2. Object Diagram
 - 3. State Machine Diagram
 - 4. Package Diagram

CONCLUSION



INTRODUCTION

The conception phase will enable us to bring out the different contours of the static model of the system in order to depict the dynamic model of our system which will help us implement our web application, all of these can be done due to the system functional needs gathered in the specification book. Firstly, we shall present the objectives of the conception and we shall proceed with the presentation of diagrams.

I. TECHNICAL BRANCH

1. Generic Design

a. Hardware diagram of the system

The hardware diagram simply shows how the system components of our system are deployed, it shows the positioning of each device into its right proportion

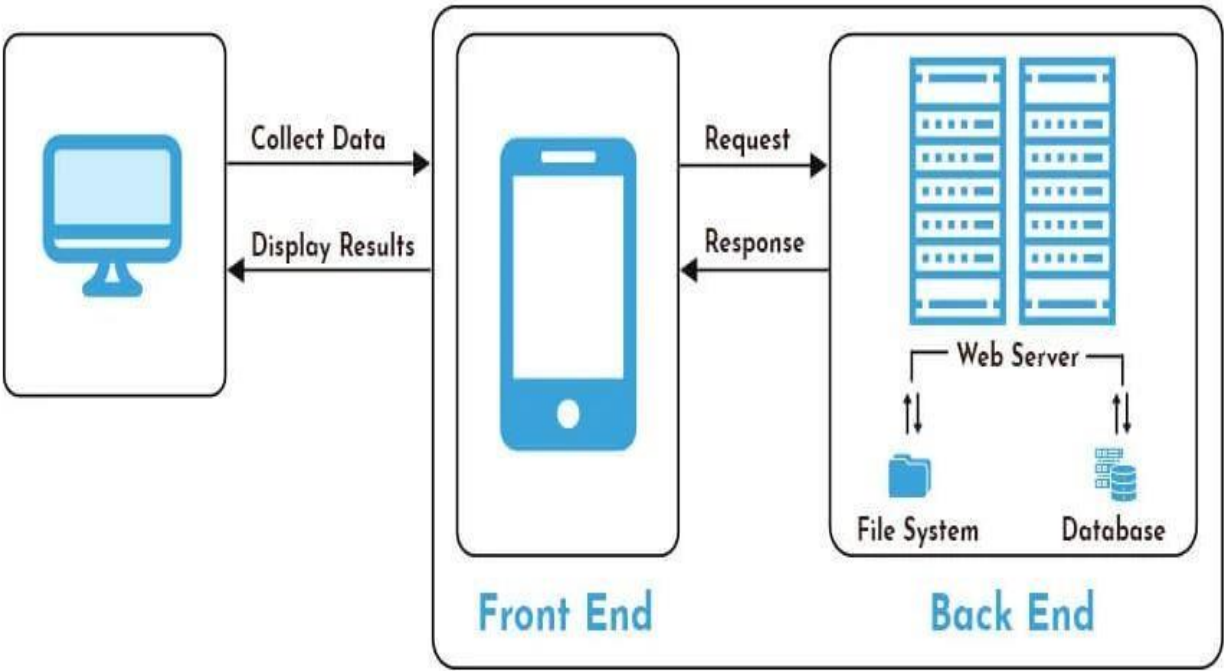


Figure 17. *Hardware diagram of system*

II. CAPTURE TECHNICAL NEEDS

1. Physical architecture

The design of the DBMS depends on its architecture. An n-tier architecture partitions of the whole system into related but separated n modules, which can be independently modified, altered changed or replace. A large amount of data on web servers, personal computers (pc) and others are link with networks with the help of basic client or server architecture.

Within the scope of our project, we made use of the multi-tier architecture. This architecture separate it tiers from each other based upon the user and the manipulated data in the database. It is important to note that with the multi-tier architecture, only neighbouring layers can communicate. Each layer has a well-defined communication interface and the evolution of the layer is independent of the other. The multi-tier of our system is made up of:

- The hardware tier, which represents our module and it is connected directly to our DBMS using a wireless connection
- The client tier, which is also known as our presentation interphase.
- Application Tier, which represents our webserver.

The data tier, which represents our DBMS server

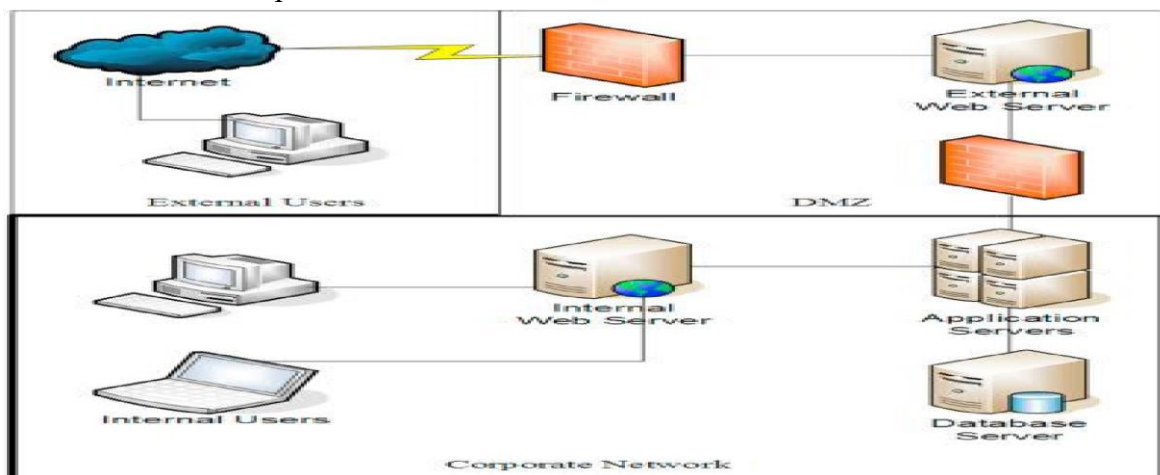


Figure 18. *Physical Architecture of system*

2. Logical Architecture

Model View controller or MVC as it is popularly called, is a software design pattern for developing application. A model view controller pattern is made up of the following three parts.

- Model: The lowest level of the pattern which is responsible for maintaining data.
- View: This is responsible for displaying all or a portion of data to the user.
- Controller: It handles software codes that controls the interactions between the model and the view. MVC is popular as it isolates the application logic from the user interface and supports separation of concerns. Here the controller receives all requests for the application then works with the model to prepare data needed by the view. The view then uses the data prepared by the controller to produce a final response. The MVC can be represented as follows

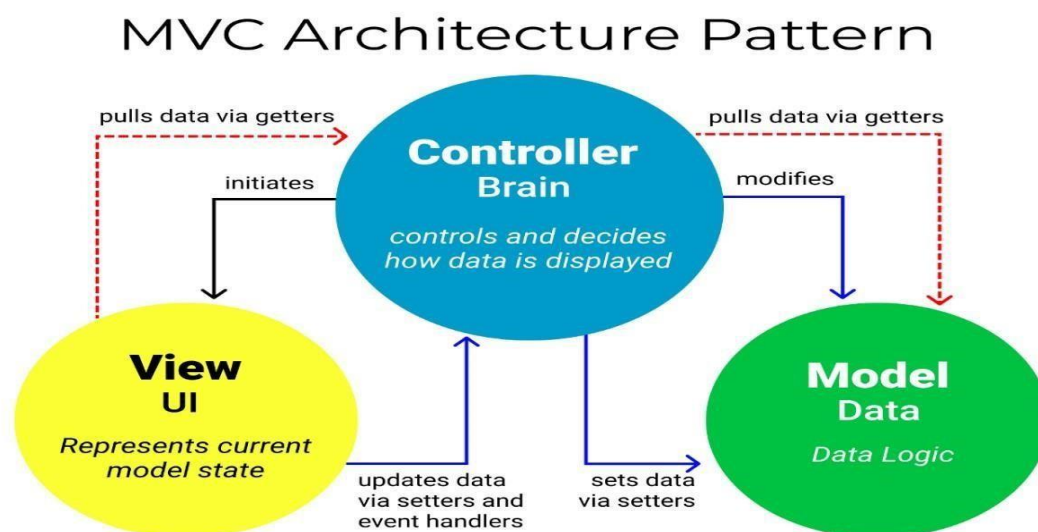
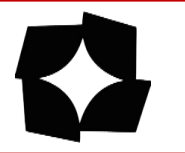


Figure 19. *MVC architecture pattern*



III. RELATED UML DIAGRAMS

1. Class Diagram

a. Definition

A class diagram is a static diagram. It is a type of structure diagram because they describe what must be present in the system being modelled. The main components in a class diagram are the building blocks of UML. A class is a type of something. We can think of a class as blueprint out of which objects can be constructed. Class diagram describes the attribute and operation of a class and constraints imposed on the system. Its purpose is to model a static view of an application. A standard class diagram is composed of the three sections (upper section, middle section and bottom section)

b. Formalism

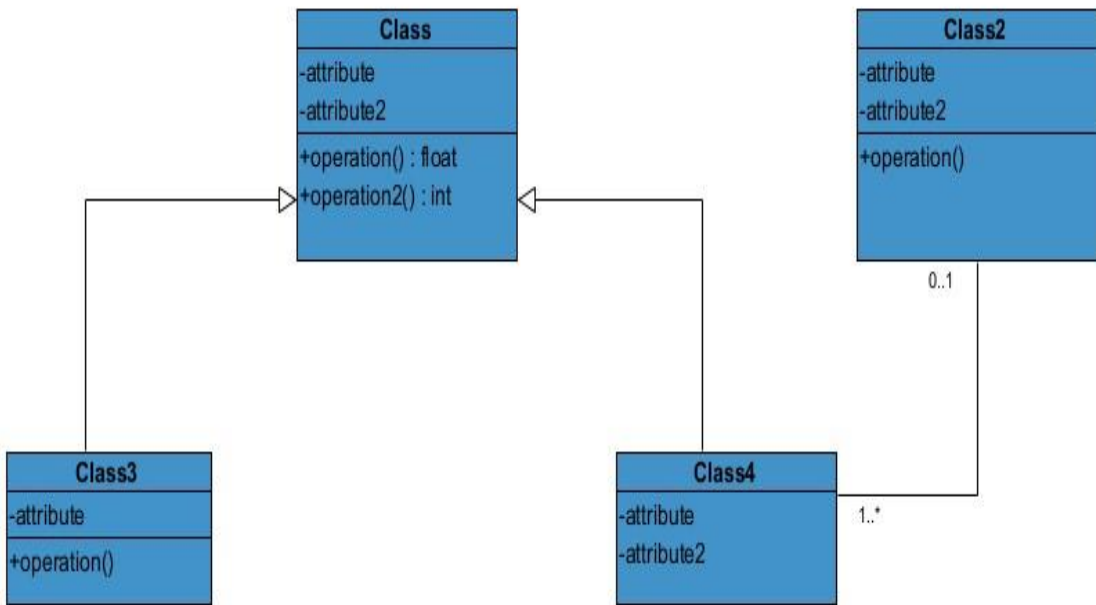


Figure 20. *Class diagram formalism*



The formalism of the class diagram is shown below

Table 20. *Formalism of class diagram*

Elements	Description	Notation
class	It is a template for creating objects and implementing behaviour in a class	
	Attributes: The second in a class shape	
	Methods: The third row in a class shape	
Generalization	It is a process of a child or sub-class taking on the functionality of a present or superclass	
Association	It is the sematic connection between two class	



Aggregation	If the parent of the aggregate is deleted, usually the children are not deleted	
Composition	If the parent of the composite is deleted, the children is also deleted	
Dependency	It is the exists between two classes, if one changes it may cause the change in the order, but the other way around	

c. System class Diagram

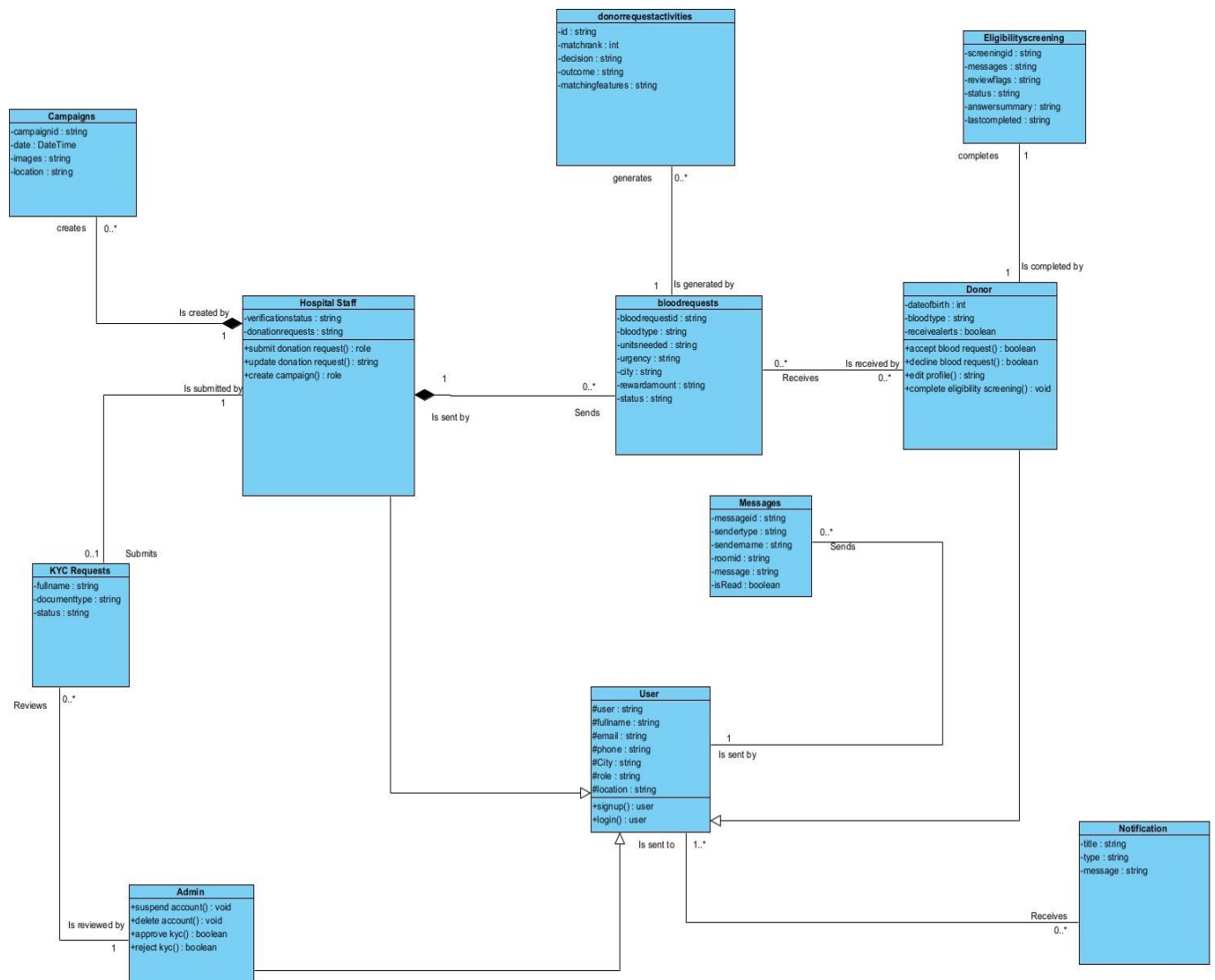


Figure 21. System Class Diagram

2. System Object Diagram

a. Definition

An object diagram is an instance of a class diagram; it shows a snapshot of the detailed state of a system at a point in time; thus, an object diagram encompasses objects and their relationships which may be considered a special case of a class diagram or a communication diagram.

b. Formalism

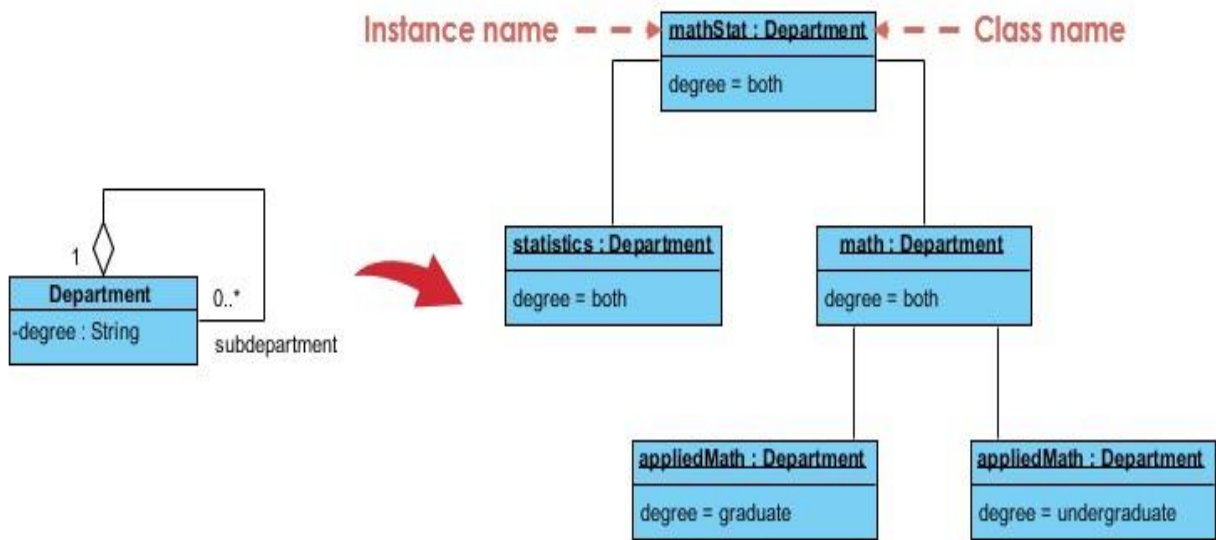


Figure 22. *Formalism of object diagram*

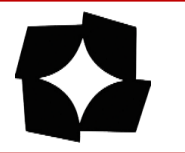
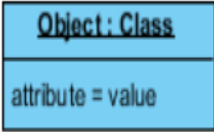
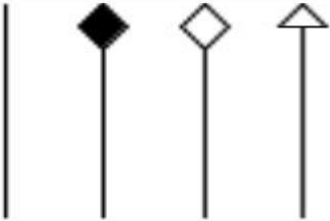


Table 21. *Formalism of object Diagram*

Element	Description	Notation
Object Name	Every object is actually symbolized like a rectangle, that offers the name from the object and its class underlined as well as divided with a colon.	
Attribute	Similar to classes, you are able to list object attributes inside a separate compartment. However, unlike classes, object attributes should have values assigned for them.	
Links	Links tend to be instances associated with associations. You can draw a link while using the lines utilized in class diagrams.	

c. System Object Diagram

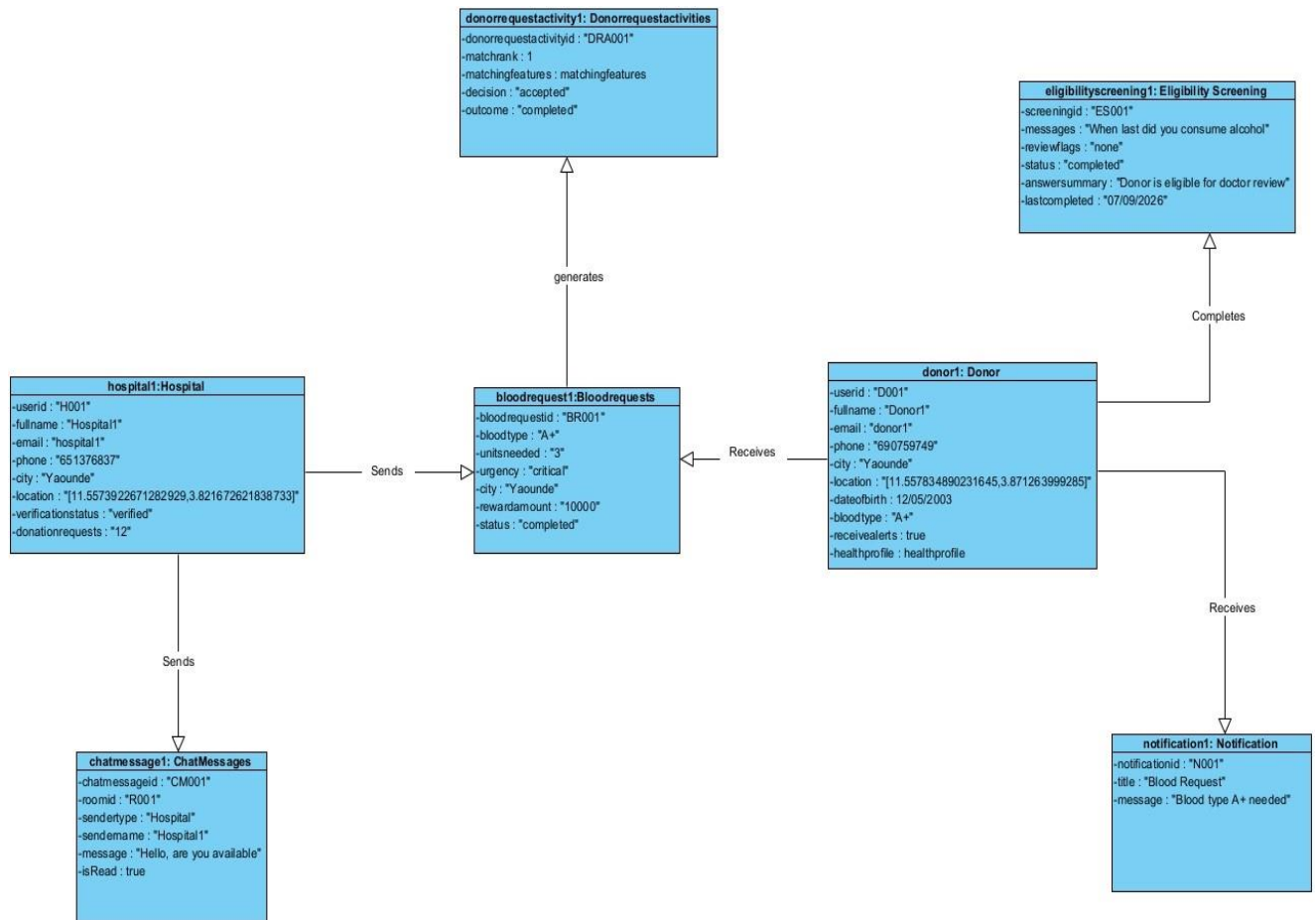


Figure 23. System Object Diagram

3. State Machine Diagram

a. Definition

A state machine diagram is a diagram that shows all the states of an entity and how it responds to various events by changing from one state to another. Its main purpose is to specify dynamic behaviours for critical objects that need to be monitored and controlled. State machine diagrams are relatively scalable.

b. Formalism

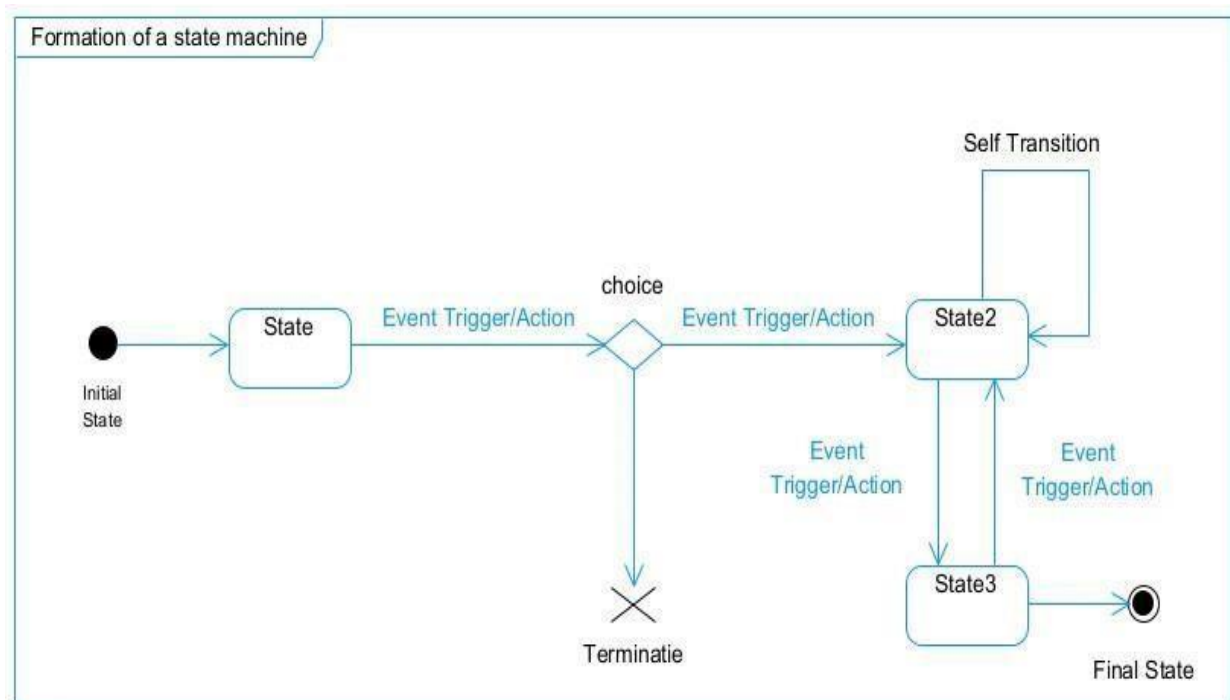


Figure 24. *State machine diagram formalism*

c. State Machine Diagram Components

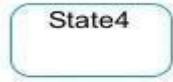
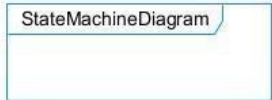
Component	Diagram Relationship	Description
State		Models a situation during which a certain invariant condition holds.
First (Initial) State		It represents a default vertex, that is, a source for a single transaction to the default or composite state.
Final State		A state specifying that the enclosing region is complete.
Transition		A direction relation between a source and a target vertex.
Choice pseudo-State		A diamond symbol that indicates a dynamic condition with branched potential results
Terminate		Implies that the execution of a state by means of it context is terminated.
Diagram Overview		A placeholder for the linked states in a state machine diagram.

Table 22. *Formalism of state machine diagram*

d. Account State Machine

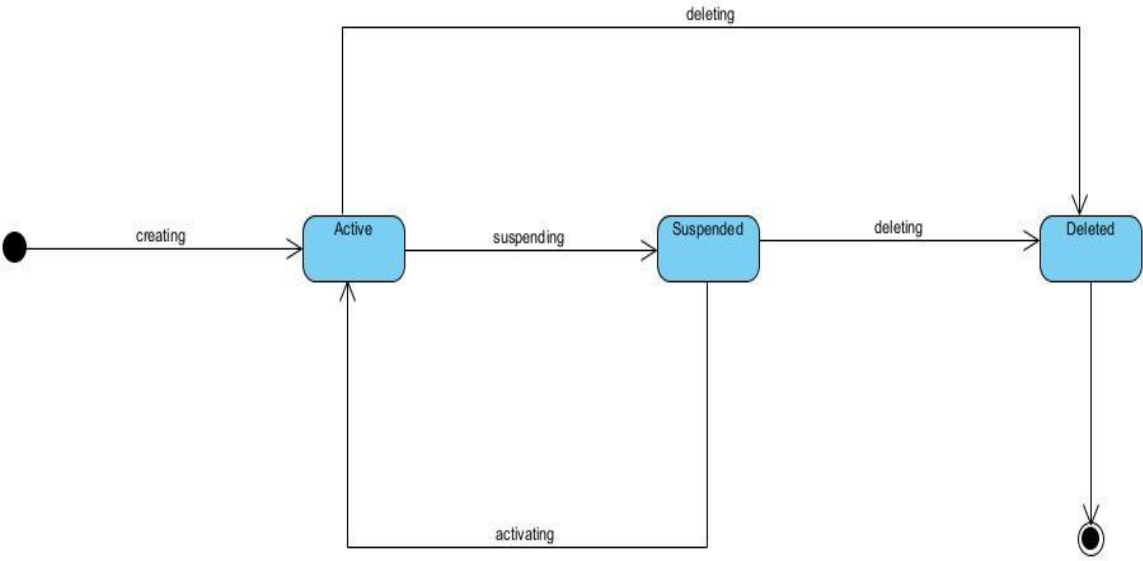


Figure 25. *Account state machine diagram*

e. Blood Request State Machine Diagram

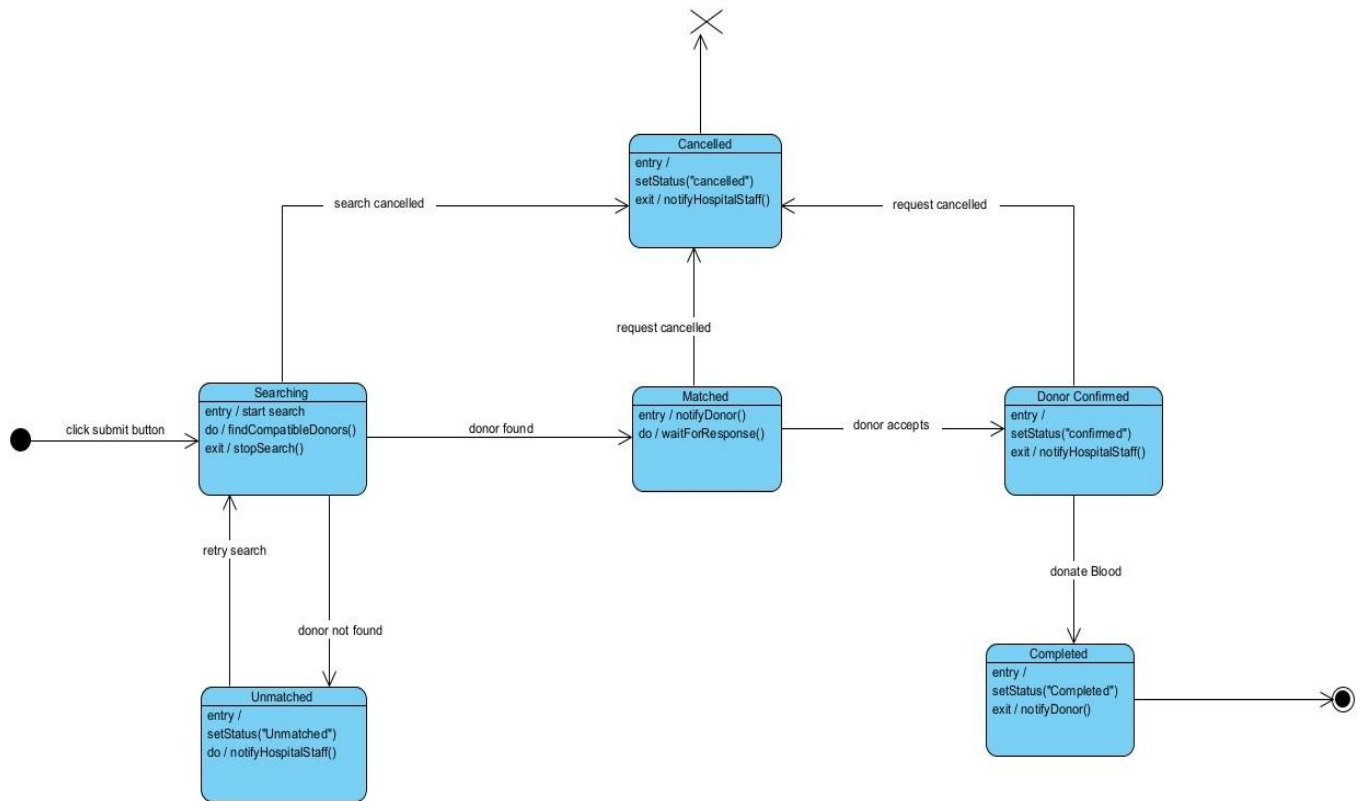


Figure 26. *Blood Request State Machine Diagram*

4. Package Diagram

a. Definition

A package diagram is a structural diagram. It is used to simplify complex classes into packages. A package is a collection of related UML elements including diagrams, documents, classes, and event packages. A package can be drawn inside the package or outside the package attached in the folder.

b. Formalism

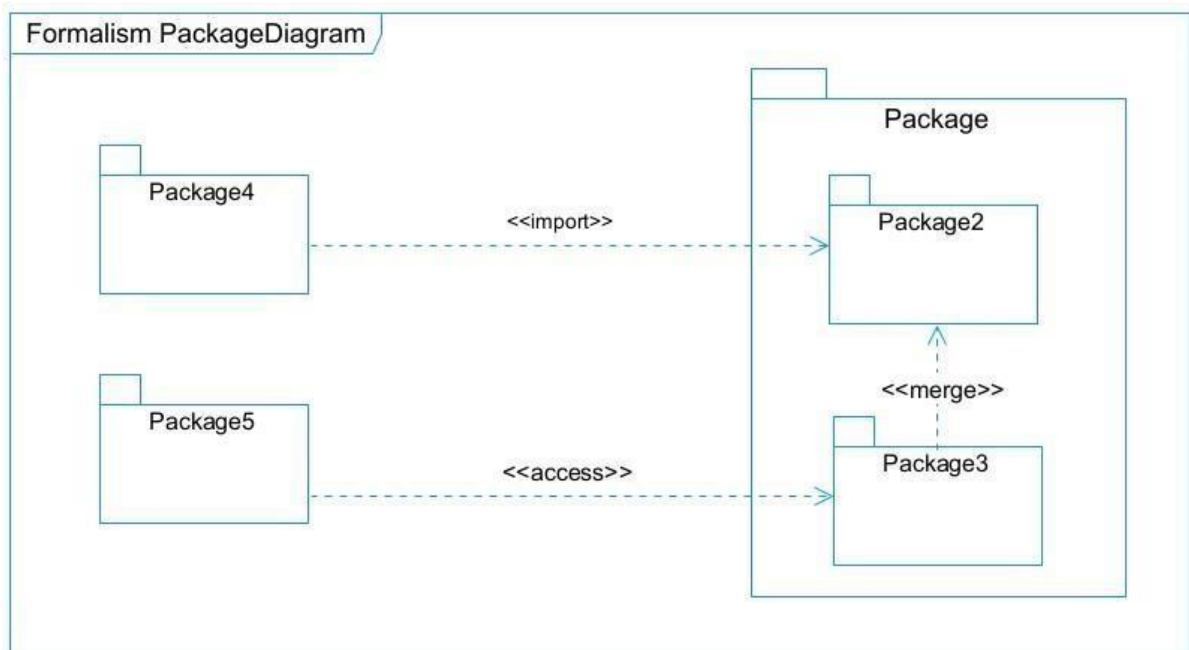
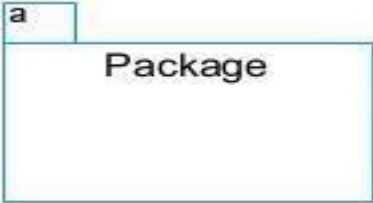
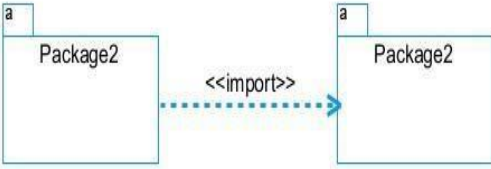
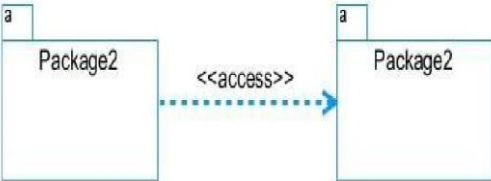
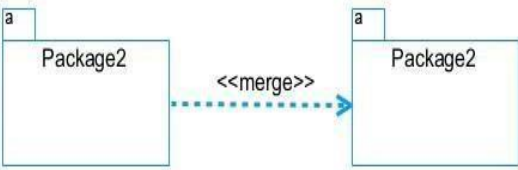


Figure 27. *Package diagram formalism*

c. Package Diagram Components

Table 23. *Formalism of Package Diagram*

Name	Representation	Description
Package		A package is a namespace use to group related elements; it is a mechanism used to group elements into a better structure in a system.
Package import		A relationship Indicate that, functionality has been imported from one package to another.
Package access		A relationship Indicates that one package requires assistance from the function of another package.
Package merge		It is a relationship which shows that, the functionality of two packages are combines to a single function.

d. System Package Diagram

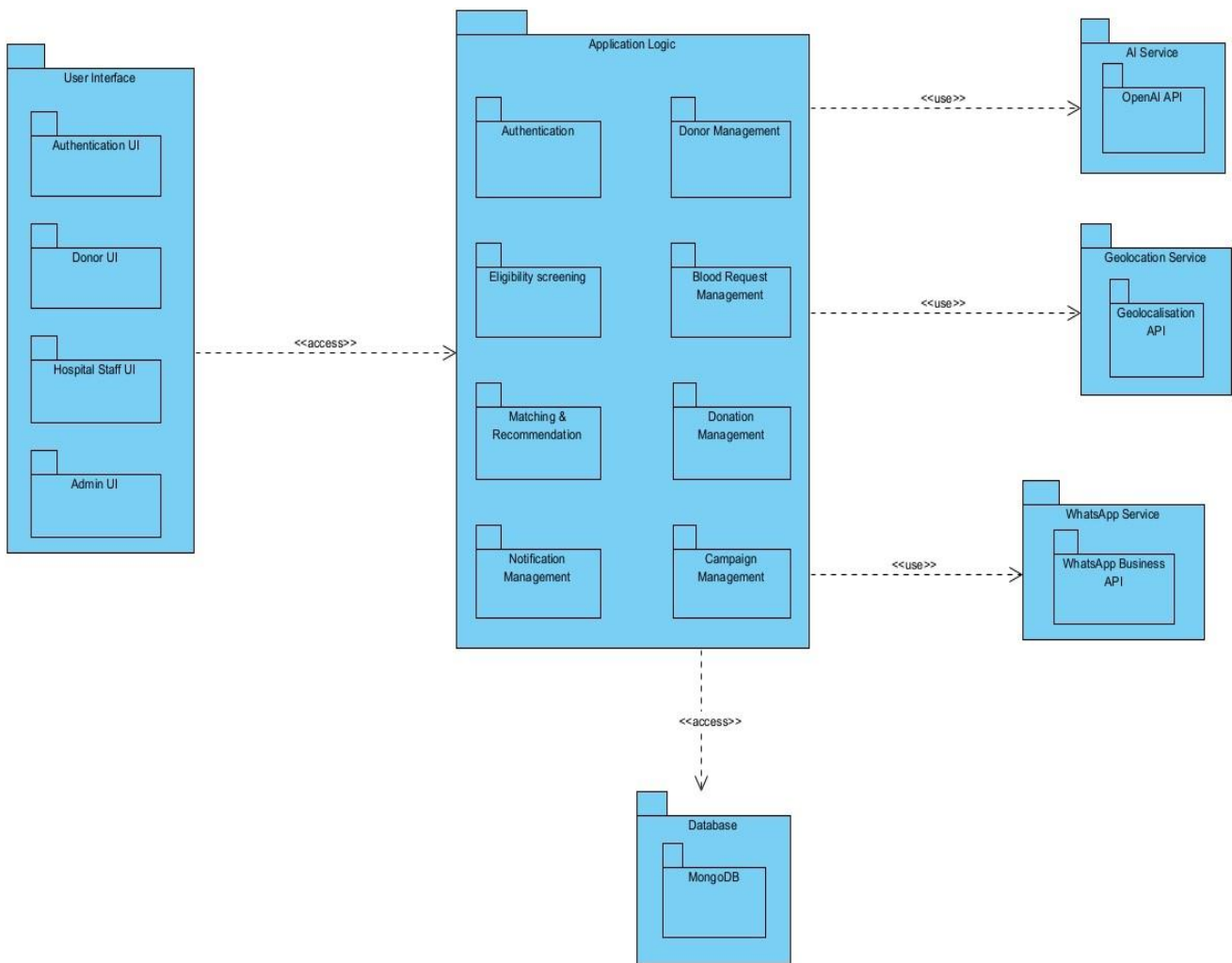


Figure 28. *System Package Diagram*



CONCLUSION

Arrived at the end of the conception phase where we outlined the participating conception steps of the system, we presented the class diagram preceded by its description. Next, we presented the state machine diagram to show the different state an object can have in the system. Finally, we presented the package diagram to illustrate the organization of elements in the system. We are now going to dive into the realization phase while taking into account the modelling elements above.



FILE V: REALIZATION FILE



Preamble

The realization phase will present technologies to be used for the realization of our project in terms of software, programming language, physical architecture and logical architecture.

Overview

INTRODUCTION

- I. ARCHITECTURE OF THE PROJECT
 - i. Logical View
 - ii. Physical View
- II. TECHNOLOGICAL CHOICE
- III. PRESENTATION OF DIAGRAMS

CONCLUSION



INTRODUCTION

The realization phase, is influence by the conception and analysis phase and deals principally with the concrete and physical domain of the project. Here, the different architectures and code use to deploy the solution is put in place by a good master of the course. We shall also see the different deployment steps and physical architecture of our database. At first, we shall present the different technologies used, next, our application architecture, then, deployment and component diagram of our application and finally, we shall present a physical data model derived from the class schema from the previous phase.



I. ARCHITECTURE OF PROJECT

1. Logical Architecture

a. MVC Logical Architecture

MVC (Model View Controller) is a logical architecture designed for graphical interfaces in 1978. It is an architecture design pattern that is used to build the applications. This architectural pattern was mostly used for web applications. It separates an application into three main logical components: The Model, the View and the Controller. Each of these components are built to handle specific development aspects of an application. MVC is one of the most frequently used industry standard web development frameworks to create scalable and extensible project. The MVC components shall be examined more detailly in the following paragraphs;

Model: It refers to the lowest part of the architecture which is responsible for maintaining data. For instance, a controller object will retrieve customer information from the database. Data is transferred between the controller components or between business logic elements. It manipulates data and sends it back to the database, or it is used to render the same information.

Vue: This part deals with the display. The view presents the way that data is presented in the application. The views are created based on the data collected from the model. By requesting information from the model, the user is presented with the output presentation.

Controller: These are those components of an application that handle user interaction. User input is interpreted by the controller, causing the model and view to change based on the information it receives. By communicating with a controller's view, a user can change the view's appearance and update the state of its associated model.

The Model View Controller Can be presented as follows

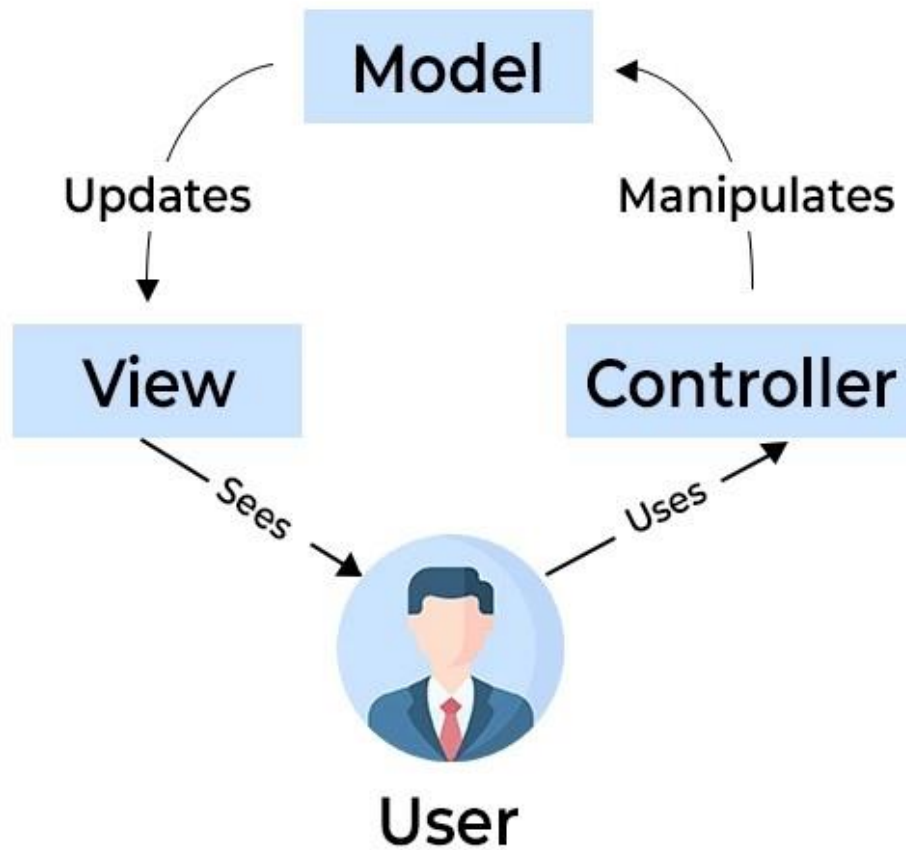


Figure 29. MVC (source: <https://th.bing.com>)

b. MVVVM Logical Architecture

MVVM (Model View ViewModel) is an architecture pattern software that facilitates the separation for the development of the graphical user interface (view) with the help of markup language from the development of the business logic or backend logic (model) so that the view is not dependent on any specific model platform. The significant component of the MVVM architecture comprises Model, View, and ViewModel which is elaborated in the section below;

- **Model:** It is referred to as an object-oriented technique which is used to denote the Realtime content or the data access layer. The content centred technique is present in the data access layer.
- **View:** The pattern of model view presenter and model view controller is displayed in the layout and structure of view and this is a content of what the end-user views on his screen. It simply holds the formatted data and essentially delegates everything to the model.
- **ViewModel:** The abstraction to view is explored by the View Model to prevent the public commands properties. The View Model is used to define the state of data provided in the model. It acts as the link between the Model and the View and makes stuff look pretty.

These three components can be represented in the diagram below;

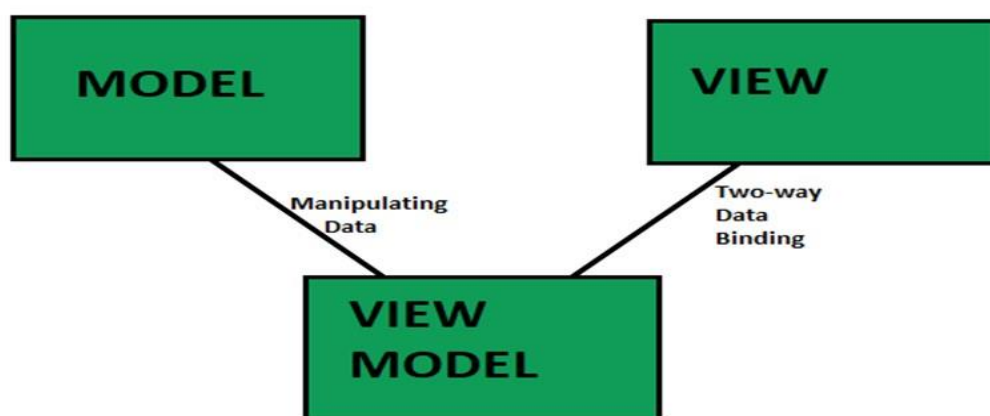


Figure 30. *MVVM (source: <https://th.bing.com>)*



2. Physical Architecture

To realize our project, we used the n-tier architecture. It refers to an architecture that partitions the whole system into related but separated n modules, which can be independently modified, altered, changed or replaced. I divide the architecture into three tiers which are: data layer, application layer and the presentation layer. These layers shall be discussed in the following paragraphs.

Presentation tier: It is the user interface and communication layer of the application, where the end user interacts with the application. Its main purpose is to display information to and collect information from the user.

Application Tier: Also known as the logic tier or middle tier, it is the heart of the application. In this tier, information collected in the presentation tier is processed (sometimes against other information in the data tier) using the business logic or a specific set of business rules.

Data Tier: Also called the database tier, data access or backend, is where the information processed by the application is stored and managed. This can be a relational database management system such as PostgreSQL, MySQL, MariaDB, Oracle, DB2, or in a NoSQL Database such as Cassandra, CouchDB or MongoDB.

In this approach, the layers communicate amongst them through an exchange model and each of them proposes a set of services. The services of a layer are available to the superior layer. Consequently, we restraints the fact that a layer can invoke the services of a lower layer (every layer doesn't communicate with no other than its immediate neighbour) The N-tier is shown in the diagram below.

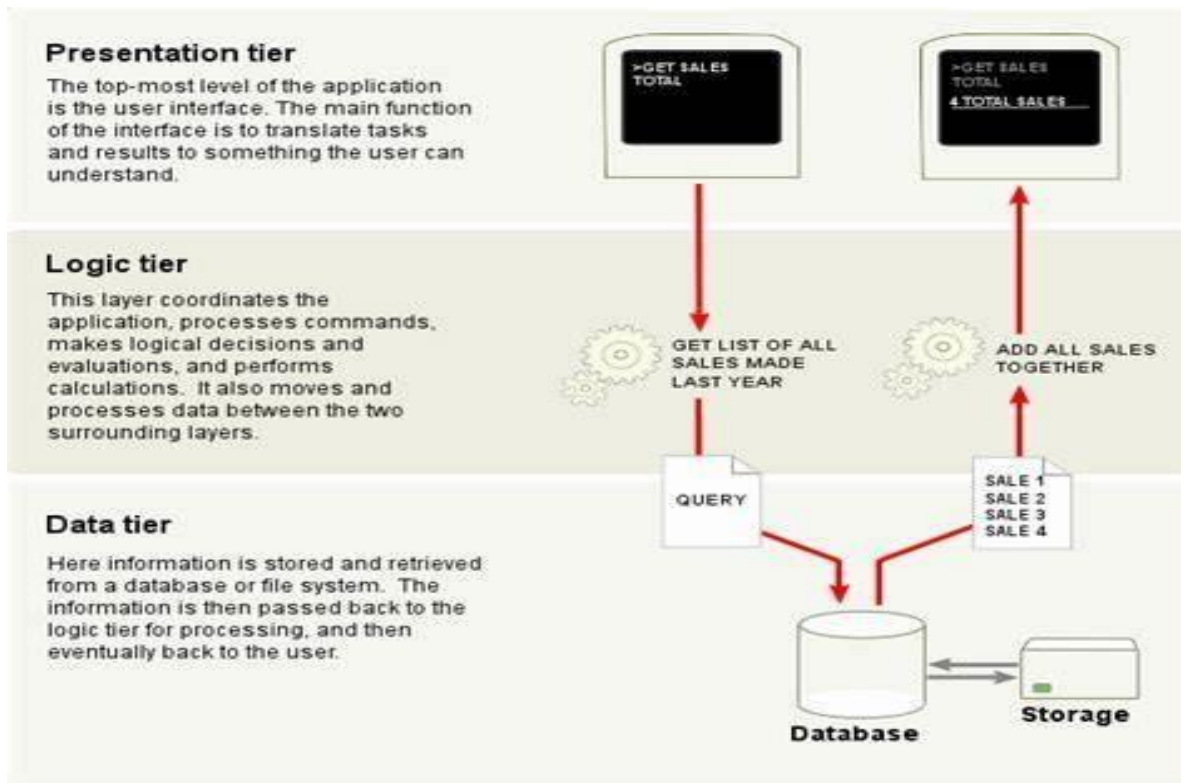
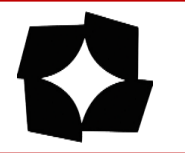


Figure 31. N-tier architecture (source: <https://th.bing.com>)



II. TECHNOLOGICAL CHOICE

Concerning the coding phase of our application, the following programming languages were used.

Table 24. *Used technologies*

Language	Description
Node js	Allows developers to create both front-end and back-end applications using JavaScript
Express	It is a node js web application framework that provides broad features for building web and mobile applications
JavaScript	It is a scripting or programming language that allows you to implement complex features on web pages
TypeScript	It is also a scripting or programming language that allows you to implement complex features on web pages
React Native	It is a free and open-sourced front-end JavaScript library for building user interfaces based on components.
Nativewind CSS	It is a utility-first CSS framework for rapidly building custom user interfaces
MongoDB	A non-relational document database providing support for JSON-like storage

III. PRESENTATION OF DIAGRAMS

1. Deployment diagram

a. Definition

The deployment diagram is a static (structure) diagram which shows the physical view of the system, bringing the software into real world by showing how software gets assigned to hardware and how the pieces communicate. It is related to the component diagram because the components are deployed using the deployment diagram. A deployment diagram consists of nodes. Nodes are nothing but physical hardware used to deploy the application.

b. Formalism

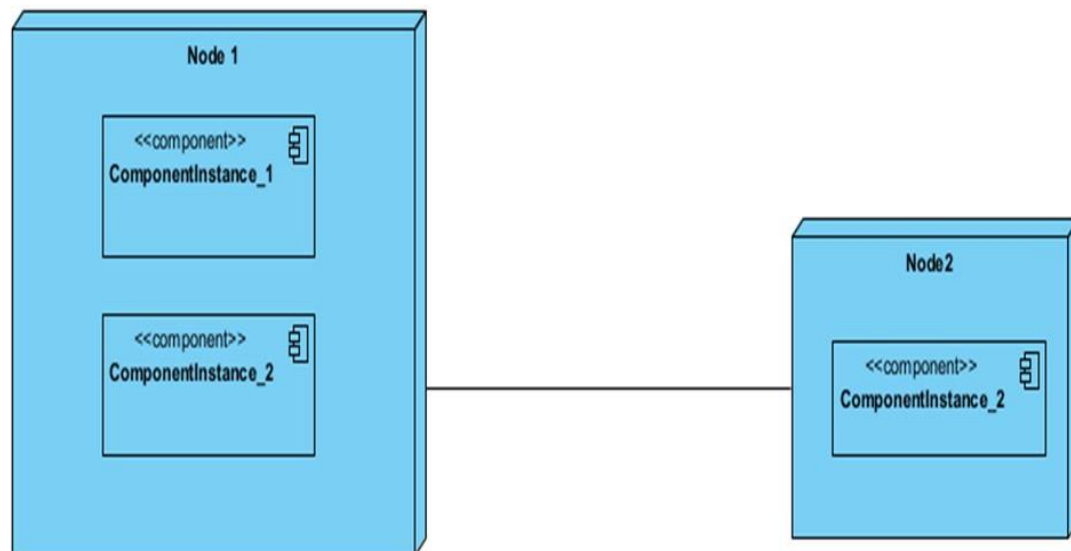
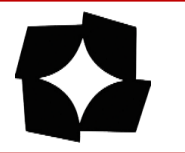
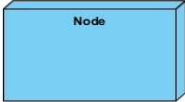


Figure 32. *Deployment diagram formalism*



The formalism of the deployment diagram is shown in the table below

Table 25. *Formalism of deployment diagram*

Elements	Description	Notation
Node	It represents the set of material tools of the system. It is represented by a cube.	
Component	Represents an entity of a system. It is represented by a rectangle with the component name	
Dependency	It is used for relations between two components. It is represented by a dotted arrow	
Association	It shows the link that exist between different nodes	

c. System deployment diagram

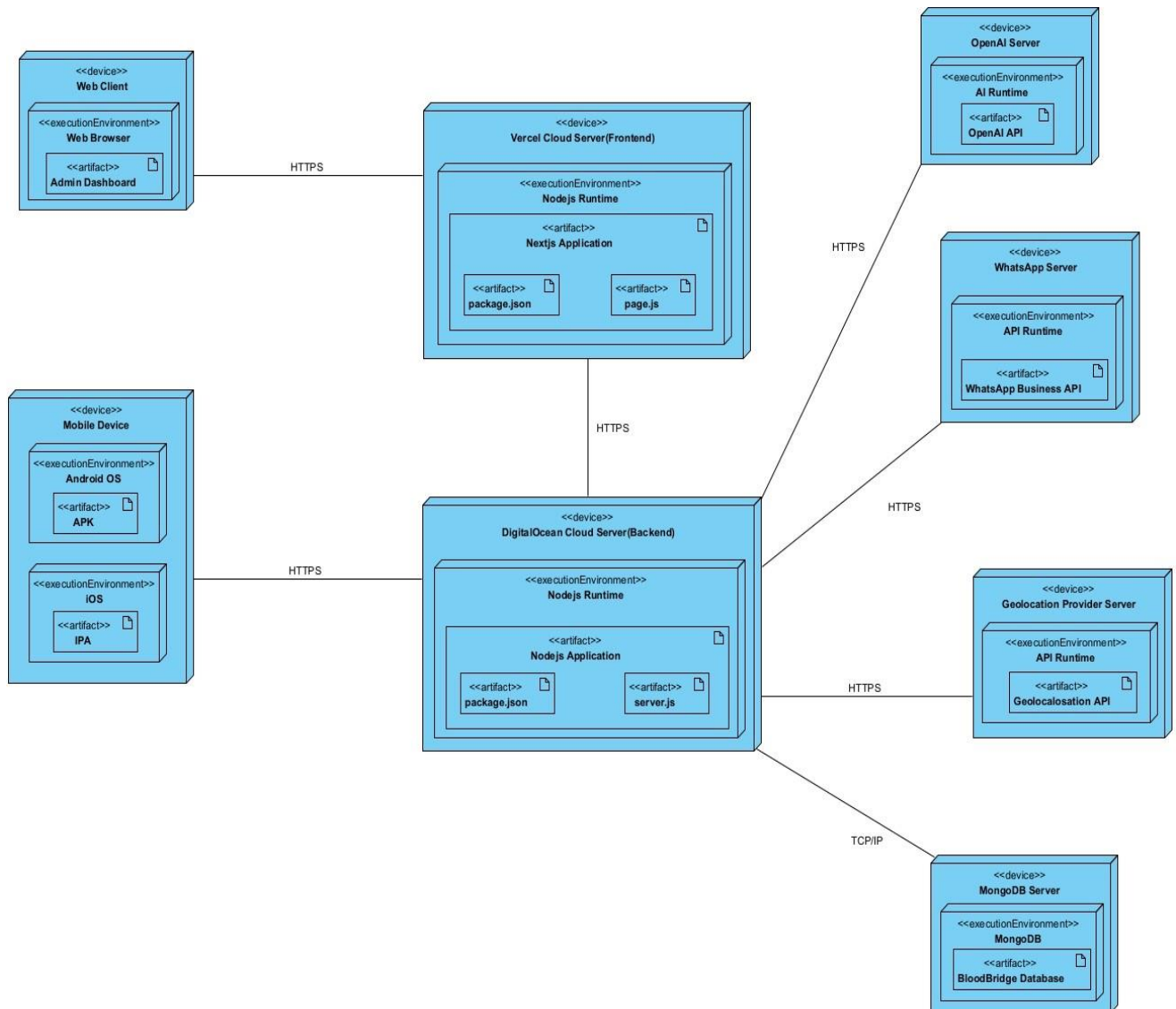


Figure 33. *System Deployment Diagram*

2. Component Diagram

a. Definition

A component diagram is use to model components in the system and as such form part of the deployment view. A component is an encapsulated, reusable and replaceable part of your software. The component diagram does not describe the functionality of the system, but it describes the components used to make those functionalities.

b. Formalism

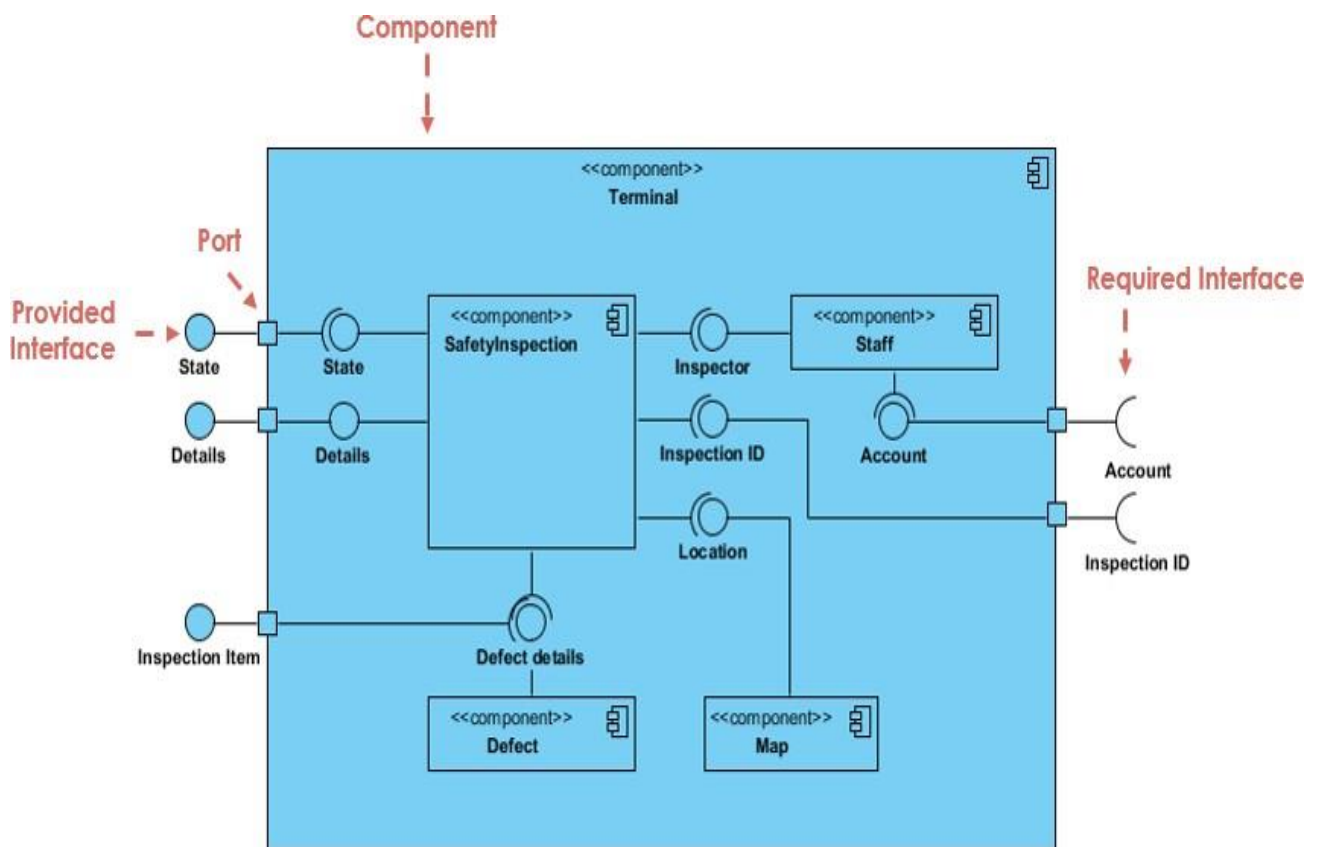
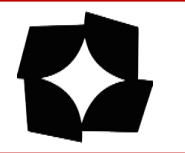


Figure 34. *Component diagram formalism*



The formalism of the component diagram is shown in the table below;

Table 26. *Formalism of a component diagram*

Element	Description	Notation
Component	It is a building block of the system. It is represented by a rectangle with the component name.	
Provided Interface	It is an interface that the components realize	
Required interface	It is an interface the components need to function	
Port	It is used to model a distinct way that the component can be used with related interface attached to the port	
Internal structure	It model parts, properties and connectors of a component	

c. System component diagram

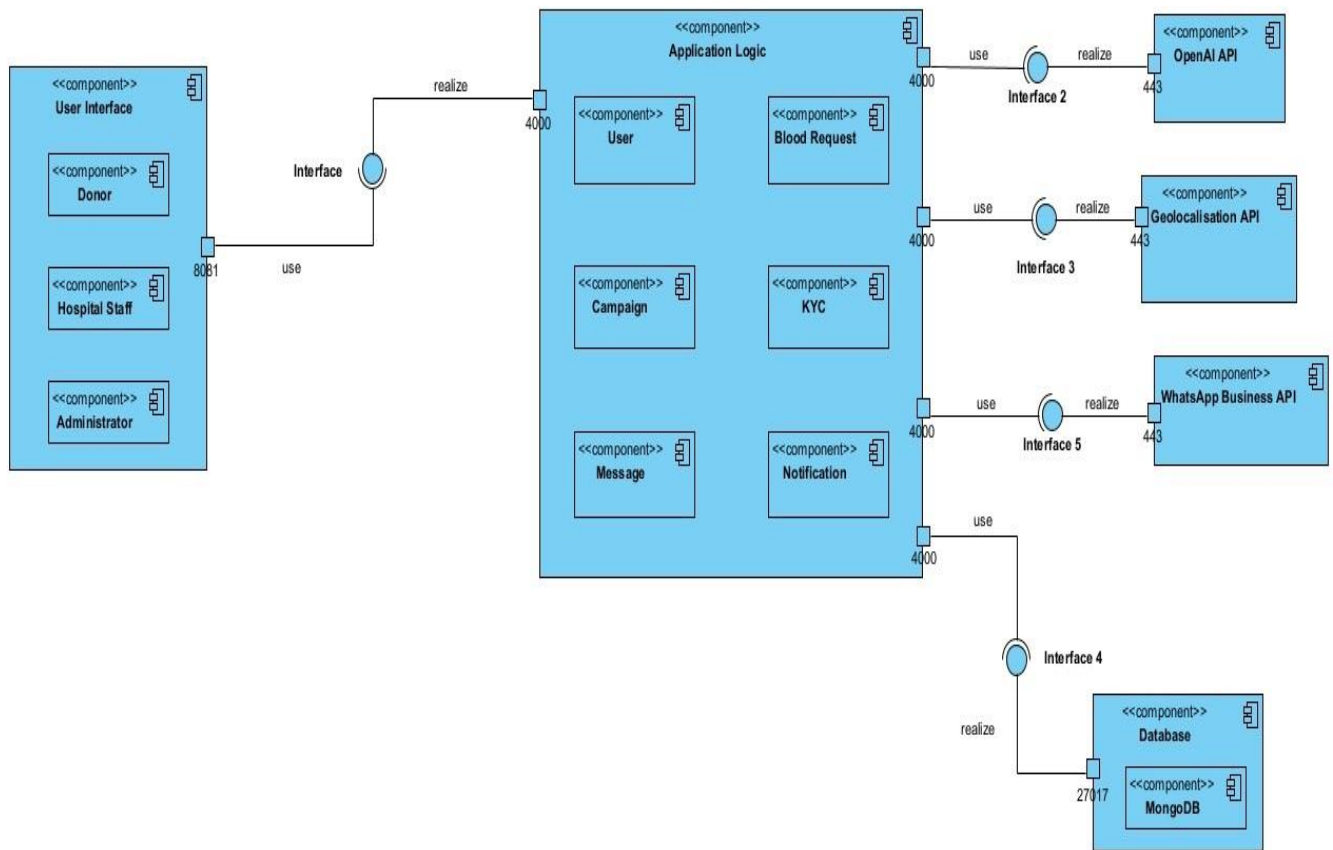


Figure 35. *System Component Diagram*

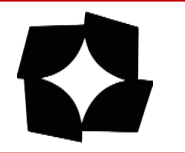


CONCLUSION

Throughout this phase, we saw how we implemented the solution presented in the previous phases. We presented the technologies used, followed by the architectures used, the component diagram, the deployment diagram of our system and the physical data model. Arrived at the end of this phase, we can dive into the user guide to help the future user of our application familiarize with it.



FILE VI: TEST OF FUNCTIONALITIES



Preamble

The test of functionalities phase illustrates the different tests made on the main functionalities of our system.

Overview

INTRODUCTION

I. FUNCTIONALITIES TEST

CONCLUSION



INTRODUCTION

The test functionalities phase helps us to know more about the solution we are building be it web or mobile. It provides the different functionalities (and test made on them) found in our application and how they are beneficial to the users. Hence, we are going to explore the different functionalities present in this project.



I. FUNCTIONALITIES TEST

The following tests were carried out using Mocha

- SUBMIT BLOOD REQUEST

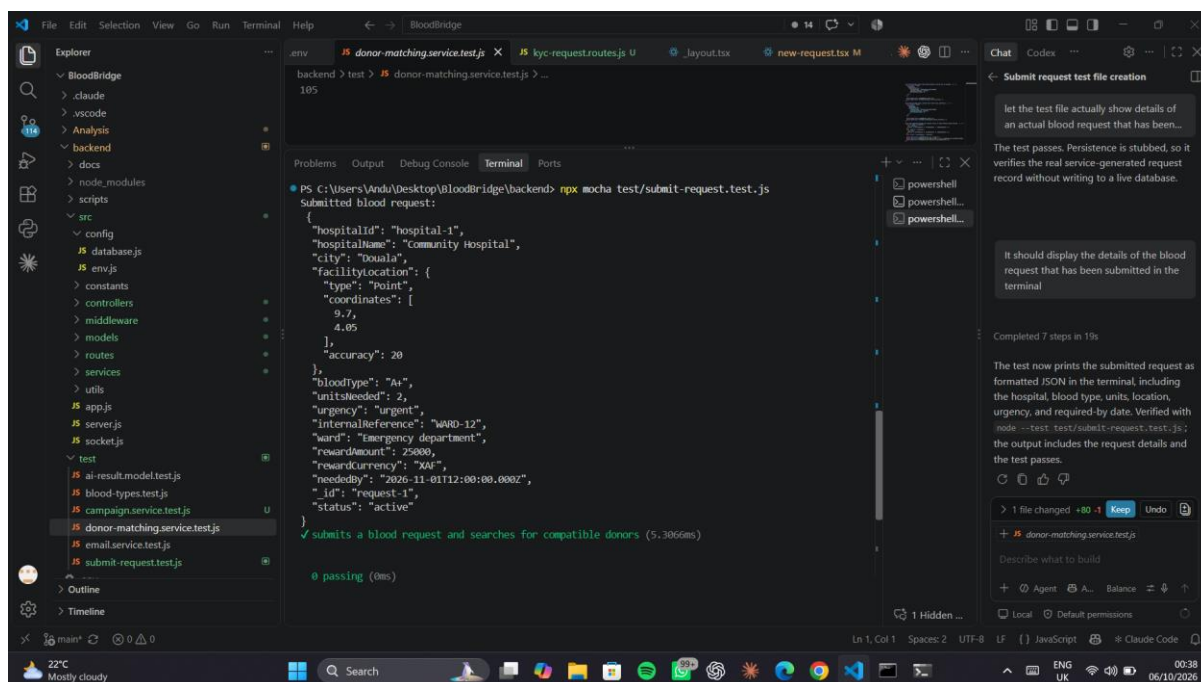


Figure 36. *Submit blood request functionality test*

- CREATE CAMPAIGN

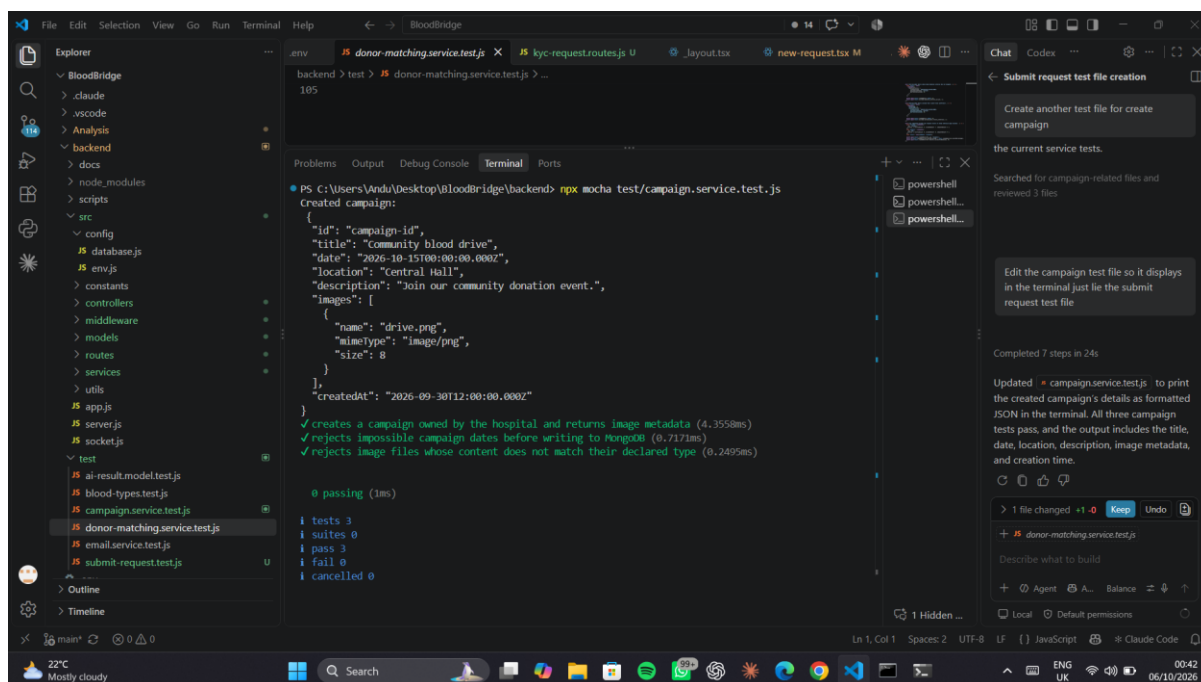


Figure 37. *Create campaign functionality test*

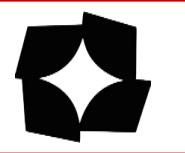


CONCLUSION

Throughout this phase, we saw the different functionalities of our system. We tested these functionalities using mocha, chai and chai http. This phase is essential in the understanding of the application. We will move to the last phase which is the installation and user guide where we will elaborate on how to install and use.



FILE VII: INSTALLATION AND USER GUIDE



Preamble

This phase includes how we acquired the resources we used for this project and how you can get them too.

Overview

INTRODUCTION

I.

INSTALLATION

II.

SHOWCASE

CONCLUSION



INTRODUCTION

This is the final phase of our report. In this phase we will walk through the requirements for our systems, the necessary installation process, accessing our system and its features, all this in a step by step manner to facilitate the setting up of the platform for the first time users. The steps of different processes will be accompanied by images. After we will showcase our application by viewing the different screens and some brief explanation.

I. INSTALLATION OF THE APPLICATION

1. Database Server Installation

To deploy a software, we need to install its database server, in our case, MongoDB will be installed in our system which is windows.



Figure 38. *MongoDB Installation*

Step 1: Downloading the MongoDB MSI installer package

We go to <https://www.mongodb.com/try/download/community> in order to download MongoDB and note that we are downloading the community version since it is free and open source.

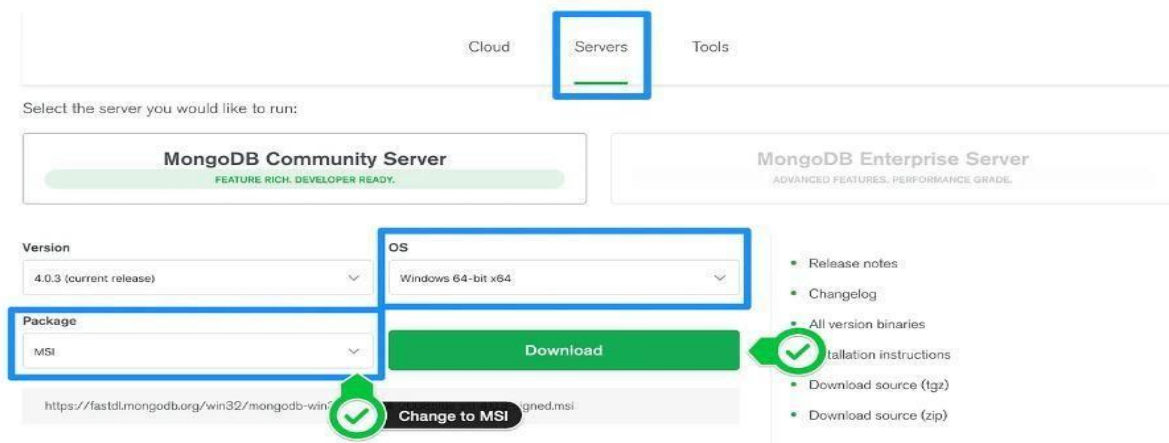


Figure 39. *MongoDB MSI Installer*

Step 2: Install MongoDB with the installation wizard

Make sure you are logged in as a user with admin privileges. Then navigate to your downloads folder and double click on the .MSI package you just downloaded. This will launch the installation wizard.



Figure 40. *Launching of Installation wizard*

Click on NEXT button



Figure 41. *Click on NEXT*

Accept the license agreement

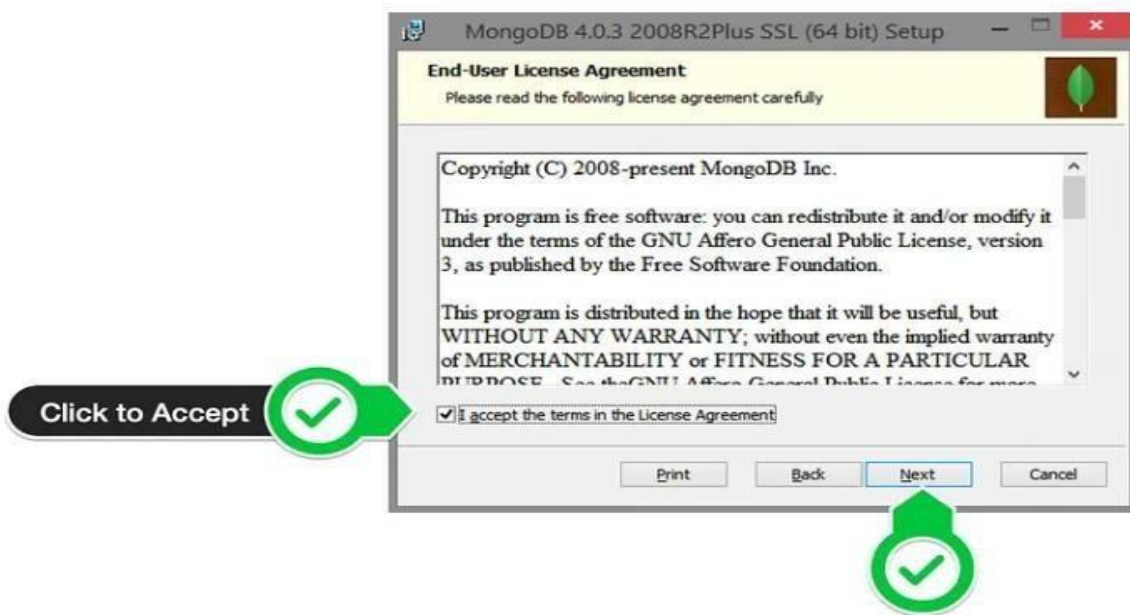


Figure 42. *License agreement*

Select complete setup

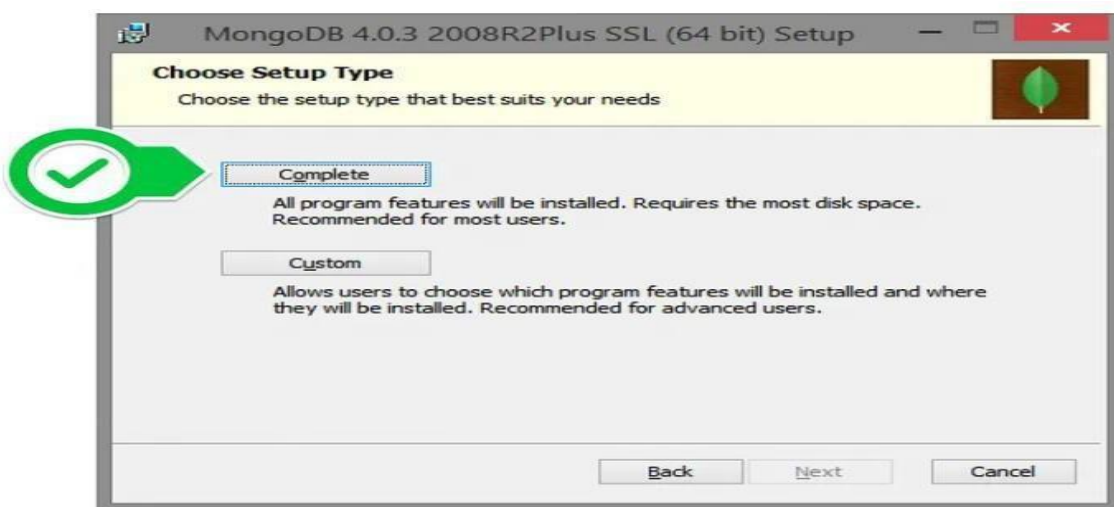


Figure 43. *Select complete setup*

Select “Run services as Network Service user” and make a note of the data directory, we’ll need this later.

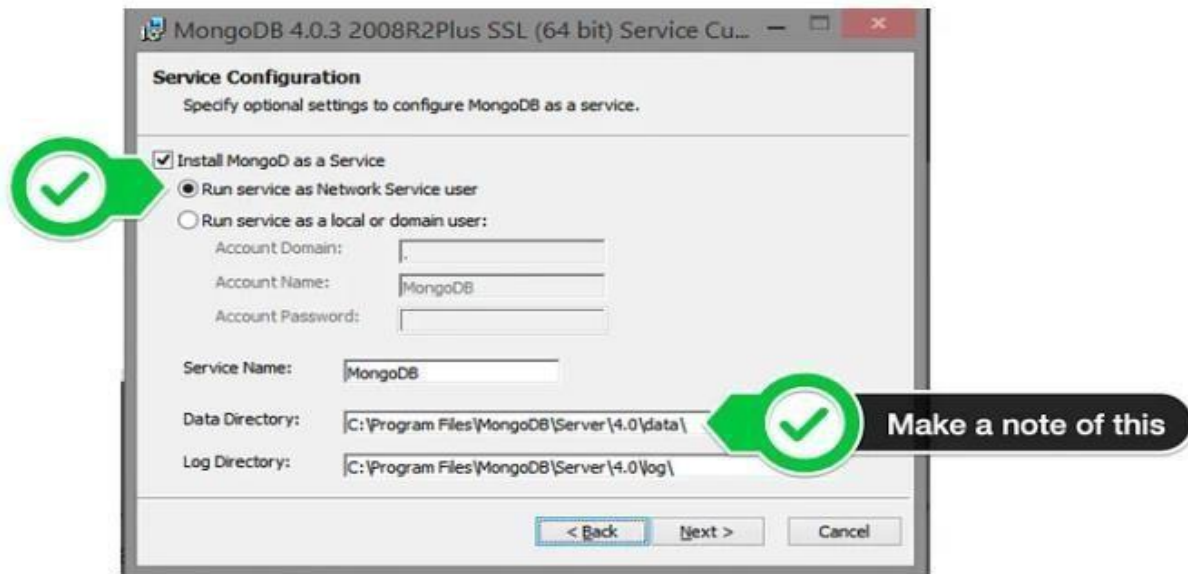


Figure 44. *Run service network user*

Select MongoDB Compass and click Next

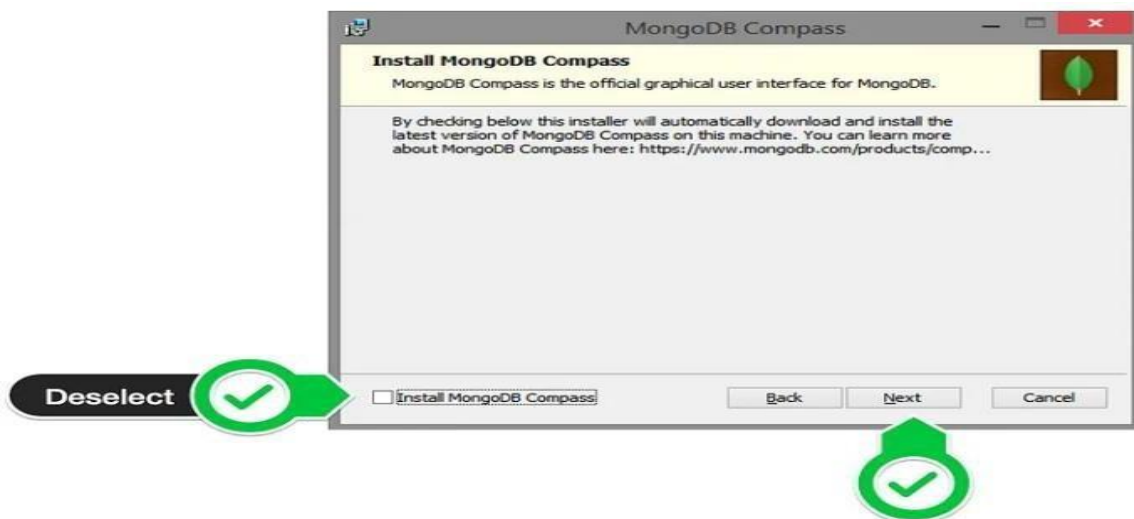


Figure 45. *Selecting MongoDB compass*

Click install to begin installation

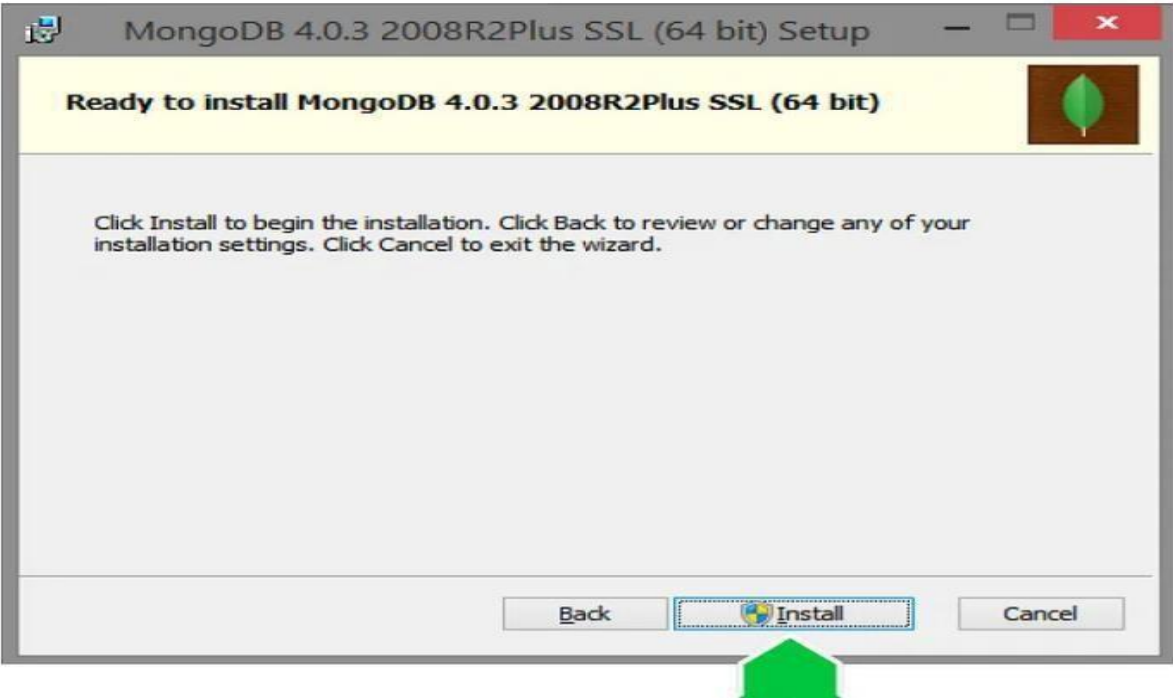


Figure 46. *Start installation*

Click on finish to complete installation

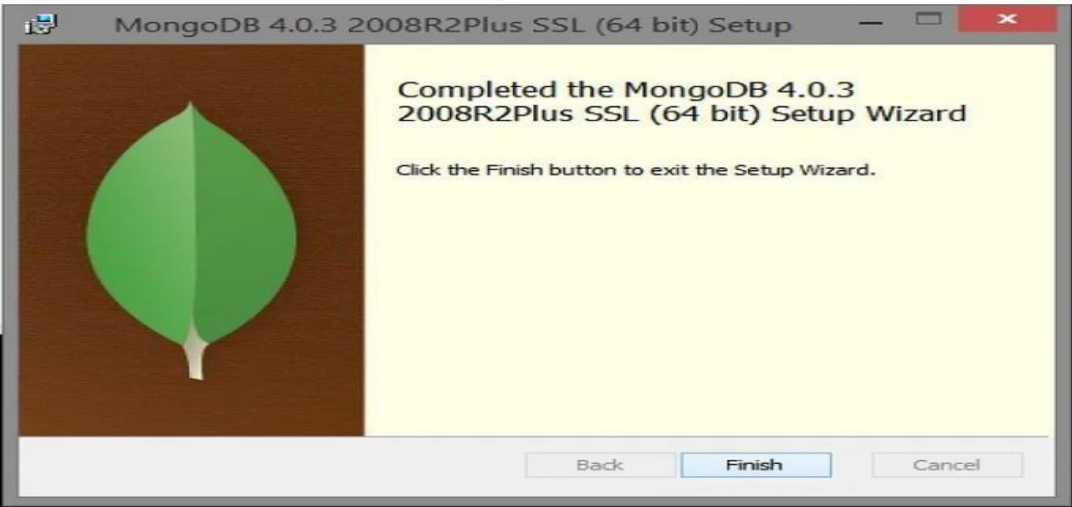
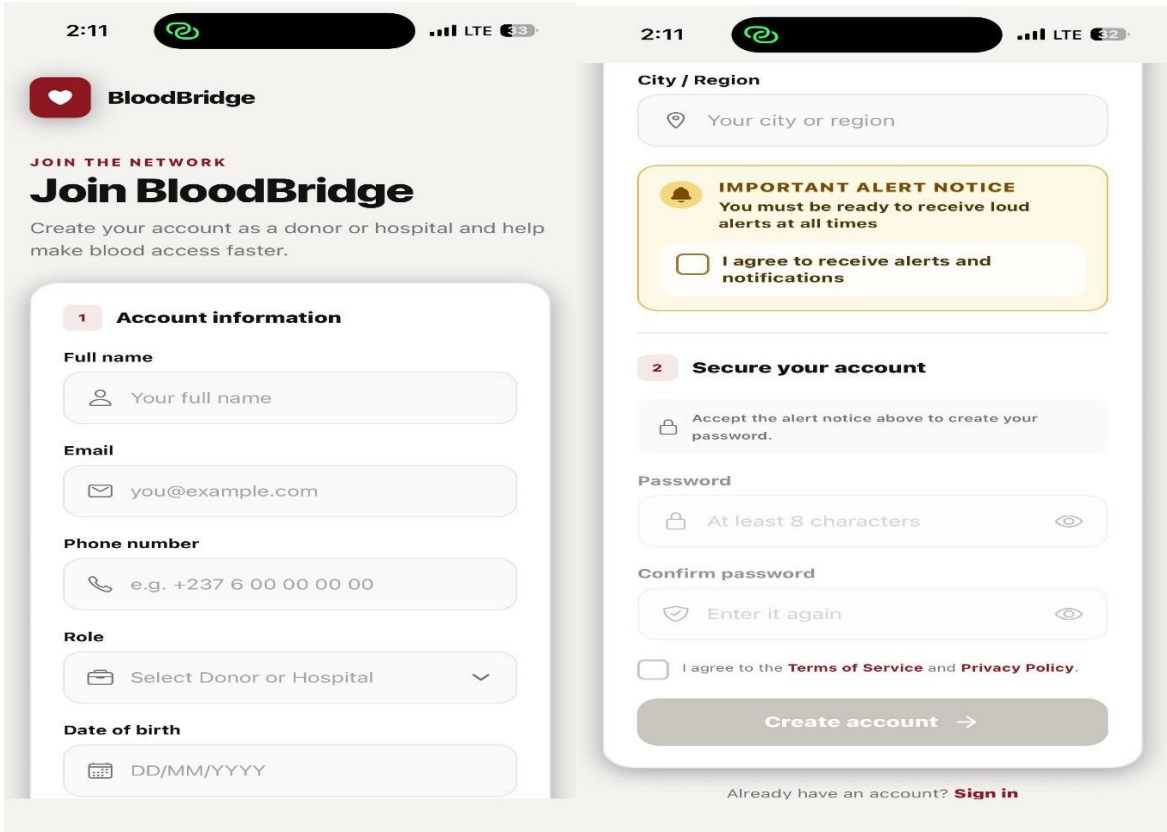


Figure 47. *Complete installation*

II. SHOWCASE

To be able to use RENTEASE, you need to have an electronic device (mobile phone, tablet, Ipad among other devices) and have access to the internet.

- Here we have the signup page



The screenshot displays the BloodBridge mobile application's signup interface. The app's logo and name are at the top left. Below it, a heading 'JOIN THE NETWORK' is followed by 'Join BloodBridge' in a larger font. A subtext explains the purpose: 'Create your account as a donor or hospital and help make blood access faster.' The form is divided into two main sections. Section 1, 'Account information', includes fields for 'Full name', 'Email', 'Phone number', 'Role' (a dropdown menu with 'Select Donor or Hospital' selected), and 'Date of birth'. Section 2, 'Secure your account', starts with a 'City / Region' field, followed by an 'IMPORTANT ALERT NOTICE' box stating 'You must be ready to receive loud alerts at all times' with an 'I agree to receive alerts and notifications' checkbox. Below this is a requirement to 'Accept the alert notice above to create your password.' The password section has fields for 'Password' (with a hint 'At least 8 characters') and 'Confirm password' (with a hint 'Enter it again'). A checkbox for 'I agree to the Terms of Service and Privacy Policy' is present. A 'Create account' button with a right arrow is at the bottom of the form. A link 'Already have an account? Sign in' is located below the button.

Figure 48. Signup page



- Here we have the login page

2:11 LTE 32%

BloodBridge

DONOR ACCESS

Welcome back

Sign in to manage your availability and respond to nearby requests.

Email address

you@example.com

Password

Enter your password

[Forgot password?](#)

Sign in →

Your donor information stays private and secure.

New to BloodBridge? [Create an account](#)

Figure 49. *Login page*



AI DRIVEN DECISION SUPPORT SYSTEM FOR INTELLIGENT DONOR MATCHING IN BLOOD DONATION NETWORKS



- Here we have the donor and hospitals dashboards.

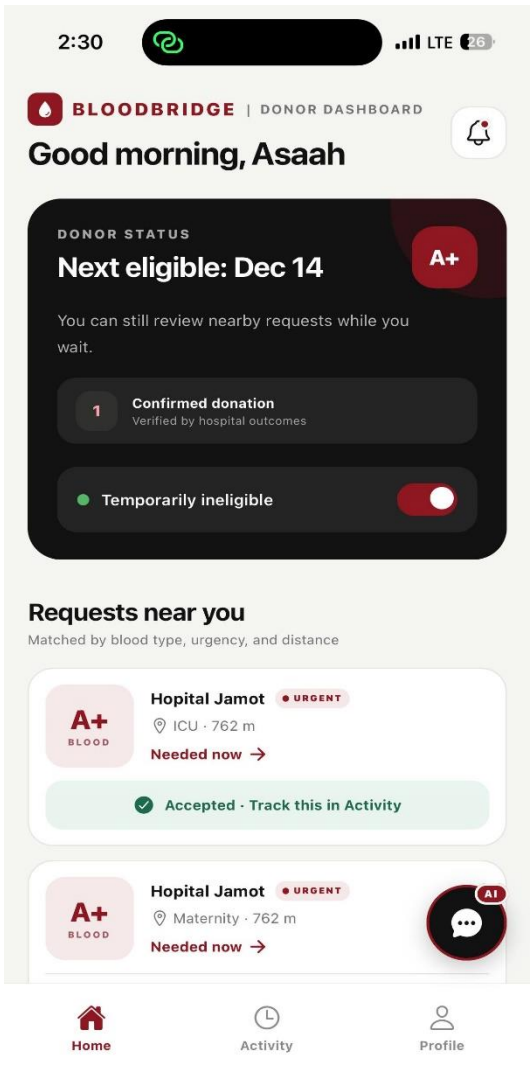


Figure 50. Donor dashboard

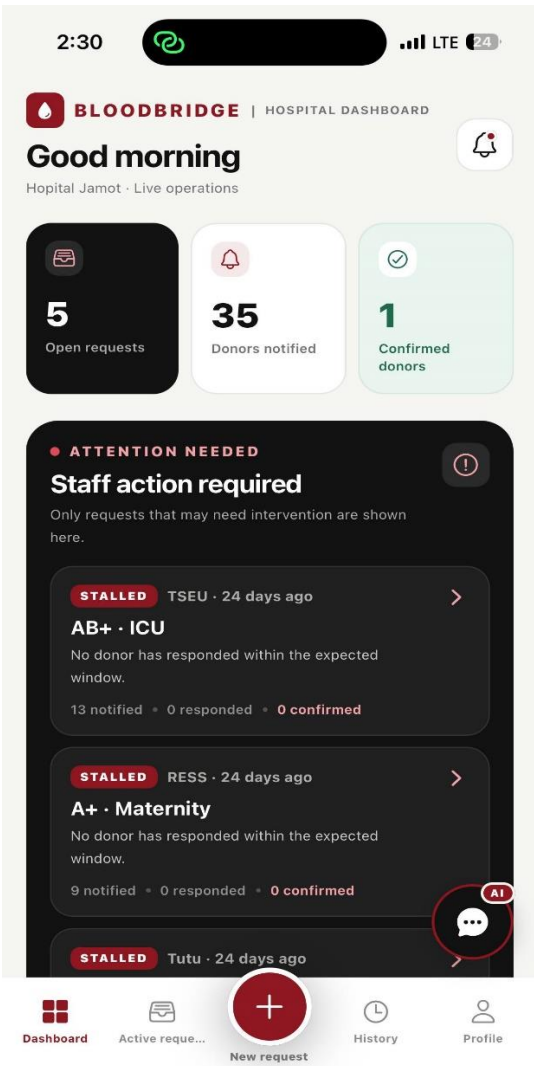
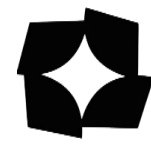


Figure 51. Hospital dashboard



- Here we have the “New request” tab

This is a page in the hospital’s dashboard. Here, the hospital can submit blood requests by filling the form and submitting the request.

HOSPITAL WORKSPACE
New request
Prepare an alert for compatible donors near your hospital.

Draft with AI
Describe the operational request in plain language. Do not include patient names, phone numbers, or medical record numbers. Details the AI cannot find stay unchanged.
Example: Critical request for 3 units O- in ICU, case BB-2482
Fill form from description

Location / facility
Hopital Jamot (AUTO-FILLED)
Yaoundé
Saved GPS confirmed · 3.8213, 11.5591
This comes from the signed-in hospital profile and will be attached to the request.

Blood type needed
A+ A- B+ B- ...

Urgency
Routine **Urgent** Critical

Internal reference
Example: Case 2482

Ward / department (OPTIONAL)
Example: East wing · ICU

Units required
Whole blood or equivalent units
- 1 +

Amount proposed (OPTIONAL)
Add a voluntary reward the hospital proposes for the donor, or leave this blank.
FCFA 0

Donor search starts after submission
Anonymous compatible donor positions appear as the expanding radius reaches them. Final rankings and totals appear when the scan finishes.

Submit request →

Bottom Navigation: Dashboard, Active requ..., **New request** (+), History, Profile

Figure 52. New request page

- Now we have the processing of the request.

When a blood request is submitted, the system carries out a search to find registered users who are compatible with the request sent.

After the search, a list of ranked donors(based on different shown reasons) is displayed and they are notified about the request.

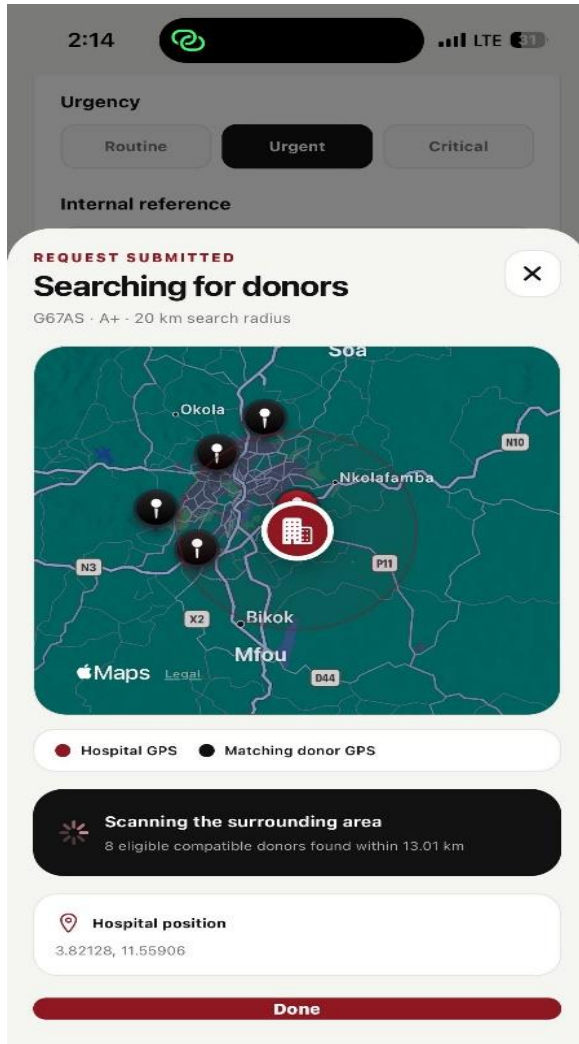


Figure 53. *Searching for donors*

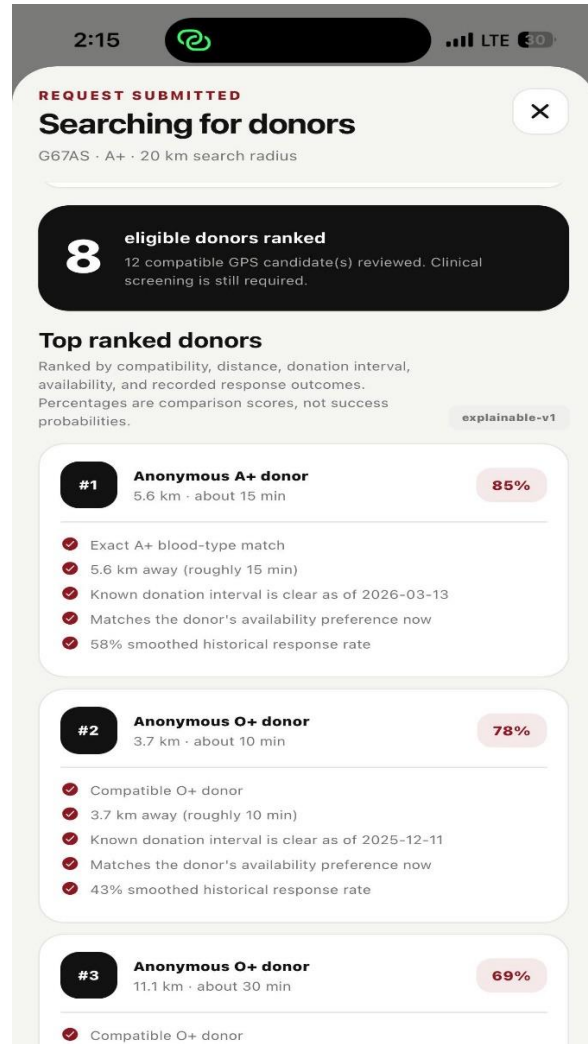


Figure 54. *Ranked donors*



CONCLUSION

Having put in place the platform, it was not sufficient for we had to produce a manual that will help its various users. That is why we presented the different tools to be installed and how they are to be installed in order to run this application without any problem and how the users will use this platform once the environment is set up



GENERAL CONCLUSION

Working on this project was both challenging and rewarding, offering valuable insights into professional teamwork and project management. We worked on the **"AI DRIVEN DECISION SUPPORT SYSTEM FOR INTELLIGENT DONOR MATCHING IN BLOOD DONATION NETWORKS"** with the goal of facilitating the process of finding donors for patients in need.

I began by analyzing the existing system and identifying limitations, then used the UML-2TUP methodology and Visual Paradigm to create system diagrams. For implementation, we utilized React Native and Expo for the frontend, NodeJS for the backend, and MongoDB for the database. The system, BLOODBRIDGE, enables hospitals/blood banks to easily submit blood requests, find donors and communicate with them.

Future updates will enhance functionality and security.



BIBLIOGRAPHY

Buku-Learning-UML2.0. April 2006, 303 pages

Mercurial 2024. 1499 pages.

Learn NodeJS in 1 day by Krishna Rungta.105 pages

Mongodb_quick_guide. 18 pages

OMG, OMG Unified Modelling Language 2.5, Object Management Group,
March 2015, 794 pages.

MongoDB course from MongoDB website 2024. Reactjs_tutorial.2023, 221 pages



WEBOGRAPHY

<https://react.dev/learn#conditional-rendering> Quick start, Tuesday September 12 2026
10:47pm

<https://react.dev/learn#conditional-rendering> React Table Component, Tuesday September
12, 2026 09:49pm

<https://react-icons.github.io/react-icons/icons?name=bs> React Icons. Monday, September 11,
2026 5:40pm

<https://react.dev/learn/managing-state> Managing state Saturday, September 9, 2026. 11:27pm
<https://react-chartjs-2.js.org/react-chartjs-2>. Saturday, September 9, 2026, 3:00pm
<https://reacthustle.com/blog/ultimate-guide-to-yup-date-validation-react-formik> Yup date
validation. Thursday, September 7, 2026, 9:59am

<https://reacthustle.com/blog/ultimate-guide-to-yup-date-validation-react-formik> MVC
Architecture. Thursday, August 31, 2026. 3:16pm

<https://www.mongodb.com/nosql-explained/data-modeling> How to design schema
from NoSQL. Monday 28 August. 10:46am

https://www.tutorialspoint.com/object_oriented_analysis_design/index.htm Object Oriented
Analysis & Design Tutorial. July 15th, 2026, 09:00 PM.

<https://icograms.com/designer> designing a geolocalisation diagram. July 31st, 2026 09:00
PM.

[https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-class-
diagram/](https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-class-diagram/) what is a Class Diagram? August 21st, 2026, 09:00 PM.

<https://formik.org/docs/guides/validation> FORMIK documentation. August 24th 2026
10:30 AM.

<https://primereact.org> Data Table. September 18th, 2026, 3:30 PM.

<https://www.mongodb.com/docs/mongoid/current/reference/associations/> Associations
September 21st , 2026, 4:30PM.



TABLE OF CONTENTS

DEDICATION.....	i
ACKNOWLEDGEMENTS	ii
CONTENT.....	iii
LIST OF FIGURES	iv
LIST OF TABLES.....	vii
ABBREVIATIONS	viii
ABSTRACT.....	ix
RESUME	x
GENERAL INTRODUCTION.....	1
PART ONE: INSERTION PHASE	2
INTRODUCTION	4
I. WELCOME AND INTEGRATION.....	5
II. GENERAL PRESENTATION OF THE COMPANY	6
1. History.....	6
2. Missions	6
3. Vision	6
4. Activities	6
5. Geographical location	7
III. ORGANIZATION OF THE ENTERPRISE.....	8
1. Administrative Organization of REALIZE.....	8
2. Functional Organization of REALIZE.....	10
IV. HARDWARE AND SOFTWARE RESOURCES OF THE COMPANY	11
1. Hardware Resources	11
2. Software Resources.....	12
V. BRIEF PRESENTATION OF THE PROJECT THEME	13
CONCLUSION.....	14
PART TWO: THE TECHNICAL PHASE	15
FILE I. EXISTING SYSTEMS	17
INTRODUCTION	19



AI DRIVEN DECISION SUPPORT SYSTEM FOR INTELLIGENT DONOR MATCHING IN BLOOD DONATION NETWORKS



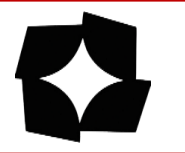
I. THEME PRESENTATION	20
II. STUDY OF THE EXISTING SYSTEM.....	20
III. CRITICISM OF THE EXISTING SYSTEM.....	22
IV. PROBLEMATIC	23
V. PROPOSED SOLUTIONS	23
CONCLUSION.....	24
FILE II: SPECIFICATION BOOK	25
INTRODUCTION	27
I. CONTEXT AND JUSTIFICATION	28
II. OBJECTIVES OF THE PROJECT	29
1. General Objective	29
2. Specific Objectives	29
III. EXPRESSION OF NEEDS	29
1. Functional needs	29
2. Non-functional needs.....	29
IV. PROJECT PLANNING.....	31
1. Chronogram of activities.....	31
2. GANTT chart.....	32
V. ESTIMATED COST OF THE PROJECT	33
1. Hardware Resources	33
2. Software Resources.....	34
3. Human Resources	35
4. Total Project Estimated Cost.....	35
VI. PROJECT CONSTRAINTS.....	36
1. List of participants	36
2. Project Deliverables	36
CONCLUSION.....	37
FILE III: ANALYSIS FILE	38
INTRODUCTION	40
I. PRESENTATION OF THE ANALYSIS METHOD	41
1. Comparative Study of UML and MERISE	41
2. Comparative study of the Unified Process.....	42
3. Justification of Method of Analysis	45
II. MODELLING OF PROPOSED SOLUTION.....	46



AI DRIVEN DECISION SUPPORT SYSTEM FOR INTELLIGENT DONOR MATCHING IN BLOOD DONATION NETWORKS



1. Use Case Diagram.....	46
2. Communication Diagram.....	54
3. Sequence Diagram	58
4. Activity Diagram.....	62
CONCLUSION.....	66
FILE IV: CONCEPTION FILE	67
INTRODUCTION	69
I. TECHNICAL BRANCH	70
1. Generic Design.....	70
II. CAPTURE TECHNICAL NEEDS	71
1. Physical architecture	71
2. Logical Architecture.....	72
III. RELATED UML DIAGRAMS.....	73
1. Class Diagram.....	73
2. System Object Diagram	77
3. State Machine Diagram.....	80
4. Package Diagram	84
a. Definition	84
CONCLUSION.....	87
FILE V: REALIZATION FILE	88
INTRODUCTION	90
I. ARCHITECTURE OF PROJECT	91
1. Logical Architecture.....	91
2. Physical Architecture	94
II. TECHNOLOGICAL CHOICE	96
III. PRESENTATION OF DIAGRAMS	97
1. Deployment diagram.....	97
2. Component Diagram.....	100
CONCLUSION.....	103
FILE VI: TEST OF FUNCTIONALITIES	104
INTRODUCTION	106
I. FUNCTIONALITIES TEST	107
CONCLUSION.....	108
FILE VII: INSTALLATION AND USER GUIDE.....	109



INTRODUCTION 111

I. INSTALLATION OF THE APPLICATION 112

1. Database Server Installation 112

II. SHOWCASE..... 117

CONCLUSION..... 122

GENERAL CONCLUSION 123

BIBLIOGRAPHYxi

WEBOGRAPHYxii

TABLE OF CONTENTS..... xiii